Royaume du Maroc — CPGE & Cycle Ingénieur

Rapport technique du projet

Plateforme de préparation au CNC

Projet 28 & extension Projet 15 (QCM adaptatif)



Ce document explique **ce qui a été fait**, **ce qu'il reste à faire** et **la logique du code** du frontend et du backend, avec des schémas et des captures d'écran de l'application.

Réalisé par : Achraf Lemrani, Mohammed Rouijel
Encadré par : M. Hosni
Référence : Cahier des charges (`docs/cdc.pdf`)
Date : 4 novembre 2026

Table des matières

1 Introduction **3**
1.1 Rappel du besoin (cahier des charges) . . . 3
1.2 Objectif de ce document . . . 3
1.3 Synthèse de l'état d'avancement . . . 4

2 Organisation du dépôt **4**
2.1 Avant et après la réorganisation . . . 4
2.2 Arborescence finale . . . 4

3 Architecture globale **5**
3.1 Vue d'ensemble . . . 5
3.2 Principes de conception . . . 6
3.3 Cycle de vie d'une requête authentifiée . . . 7

4 Frontend : ce qui a été fait **7**
4.1 Pile technique et volume . . . 7
4.2 Routes et navigation . . . 8
4.3 Écrans réalisés . . . 9
4.3.1 Authentification (M1) . . . 9
4.3.2 Dashboard (M5) . . . 10
4.3.3 Cours et exercices en double mode (M2) . . . 11
4.3.4 Annales CNC et simulateur d'épreuve (M3) . . . 14
4.3.5 Problèmes, QCM et contact . . . 17
4.3.6 Back-office et espace personnel . . . 18
4.4 Bilan du frontend . . . 20

5 Backend : ce qui a été fait **20**
5.1 Organisation en monorepo . . . 20
5.2 Le paquet partagé `@cnc/common` . . . 21
5.3 L'API Gateway . . . 22
5.3.1 Chaîne de traitement d'une requête . . . 22
5.3.2 Table de routage . . . 22
5.4 auth-user-service (Sprint 1, implémenté) . . . 23
5.4.1 Découpage en couches . . . 23
5.4.2 Modèle de données (schéma `auth`) . . . 23
5.4.3 Endpoints . . . 24
5.4.4 Stratégie des jetons . . . 24
5.4.5 Rotation des refresh tokens et détection de vol . . . 24

5.4.6 Mesures de sécurité . . . 26
5.5 Les cinq autres microservices (squelettes) . . . 26
5.6 Logique métier déjà codée : Streak et QCM adaptatif . . . 27
5.6.1 Moteur de Streak (`analytics-streak-service/src/streak.ts`) . . . 27
5.6.2 Moteur QCM adaptatif (`adaptive-qcm-service/src/adaptive-engine.ts`) 28
5.7 Vérifications réalisées . . . 28

6 Ce qu'il reste à faire 29
6.1 Backend : sprints restants . . . 29
6.1.1 Flux d'événements prévu pour le Streak . . . 30
6.2 Frontend : branchement sur l'API . . . 30
6.2.1 Étape 1 : un client HTTP unique . . . 30
6.2.2 Étape 2 : remplacer les données en dur . . . 31
6.2.3 Étape 3 : harmoniser les énumérations . . . 32
6.3 Écarts restants avec le CDC . . . 32
6.4 Dette technique du frontend à traiter . . . 32
6.5 Planning proposé . . . 33

7 Guide de démarrage 33
7.1 Prérequis . . . 33
7.2 Backend . . . 33
7.3 Frontend . . . 33
7.4 Ajouter une route dans un service (méthode) . . . 34

1 Introduction

1.1 Rappel du besoin (cahier des charges)

Le cahier des charges (CDC) décrit une plateforme web de préparation au **Concours National Commun** (CNC) pour les étudiants de CPGE des filières **MP**, **PSI** et **TSI**, sur quatre matières : Mathématiques, Physique, Sciences de l'Ingénieur et Informatique. Les modules fonctionnels sont :

M1 Authentification, profil étudiant et navigation libre Sup / Spé (JWT + bcrypt) ;

M2 Cours par leçon et exercices en double mode (PDF classique et mode interactif type Bibm@th) ;

M3 Problèmes, annales CNC 2010–2026, simulateur chronométré et « Mes Problèmes Rédigés » ;

M4 Bibliothèque d'ouvrages de référence ;

M5 Dashboard personnel : Streak, heatmap, statistiques par matière ;

M6 Contact admin et signalement d'errata (tickets) ;

M7 Fonctionnalités optionnelles (fiches express, TIPE, simulateur de rang, oraux, carnet d'erreurs).

L'extension **Projet 15** ajoute un QCM intelligent : indice de maîtrise $M(u, c)$, difficulté adaptative et répétition espacée.

Sur le plan technique, le CDC (section 3.1) impose une **architecture microservices en Node.js (TypeScript)** : une API Gateway et six microservices, PostgreSQL 16 avec Prisma (un schéma logique par service), Redis et un stockage objet S3/MinIO.

1.2 Objectif de ce document

Ce rapport sert à relire et comprendre le projet. Il répond à trois questions :

Ce qui a été fait
l'organisation du dépôt en deux applications séparées, le frontend (prototype complet) et le backend (architecture complète, Sprint 1 implémenté et testé).

Ce qu'il reste à faire
les sprints 2 à 5 du backend, le branchement du frontend sur l'API et les écarts au CDC.

La logique du code
le cheminement d'une requête, le rôle de chaque fichier et les algorithmes (JWT, rotation des refresh tokens, Streak, QCM adaptatif).

1.3 Synthèse de l'état d'avancement

Élément	Contenu	État
Organisation du dépôt	`frontend/`, `backend/`, `docs/`, `archive/` séparés	**Fait**
Frontend (prototype)	7 routes, 42 composants, environ 37 700 lignes, données de démonstration	**Fait**
API Gateway	Routage, JWT, rate limiting, CORS, santé agrégée	**Fait**
`auth-user-service`	Inscription, connexion, refresh avec rotation, déconnexion, profil, liste admin	**Fait**
Package `@cnc/common`	Configuration, erreurs, contexte d'identité, application type	**Fait**
5 autres microservices	Schéma Prisma complet, routes déclarées (réponse 501)	**Partiel**
Moteur de Streak, moteur QCM	Formules du CDC codées et testées (logique pure)	**Fait**
Logique métier des sprints 2 à 5	Catalogue, examens, analytique, contact, QCM	**À faire**
Branchement frontend ↔ API	Remplacer les données en dur par des appels HTTP	**À faire**

Table 1 – État d'avancement global

2 Organisation du dépôt

2.1 Avant et après la réorganisation

Avant la réorganisation, le frontend Next.js occupait la racine du dossier `projet/`. Il y côtoyait un dossier `backend/` incomplet : son `package.json` appelait `gateway.ts` et `auth.ts`, deux fichiers qui n'existaient pas. À la même racine se trouvaient aussi le CDC, des captures, des archives zip, un dossier `tmp/` de 580 Mo et une jonction Windows (`test_junc`) qui pointait vers `app/`.

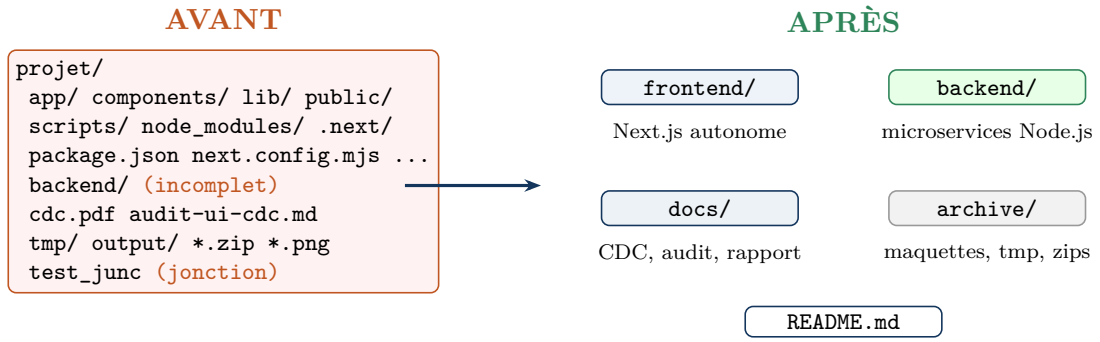


Figure 1 – Réorganisation du dépôt en deux applications indépendantes

2.2 Arborescence finale

Listing 1 – Arborescence de premier niveau

```
projet/
+-- README.md                Explique l'organisation et les commandes
+-- frontend/                Application Next.js (package.json et node_modules propres)
|   +-- app/                 Routes App Router : /, /login, /signup, /admin, /pochettes, /
    parametres
|   +-- components/          42 composants React (dont document-studio/ et ui/)
|   +-- lib/                 auth.ts (session), bases d'annales, manifestes JSON
|   +-- public/              579 PDF (annales, problèmes), pdf.js, logos
|   +-- scripts/             téléchargement d'annales, génération de PDF, logos
+-- backend/                 Monorepo npm workspaces (package.json racine)
|   +-- gateway/             API Gateway (port 4000)
```

```
|   +-- packages/common/      code partagé par tous les services
|   +-- services/             6 microservices (ports 4001 à 4006)
|   +-- compose.yaml          PostgreSQL 16, Redis 7, MinIO
|   +-- .env.example          variables d'environnement documentées
+-- docs/                     cdc.pdf, audit-ui-cdc.md, test-accounts.md, rapport/
+-- archive/                  anciens fichiers, conservés mais non utilisés
```

Ce qui garantit l'indépendance des deux applications

- Chaque application a ses propres `package.json`, `node_modules` et `tsconfig`. On peut travailler sur l'une sans installer l'autre.
- Elles communiquent *uniquement* en HTTP/JSON, à travers la gateway (`http://localhost:4000/api/...`).
- Après le déplacement, le build de production du frontend (`next build`) réussit, et les scripts du frontend se repèrent par rapport à leur propre dossier (`import.meta.url`).

Points d'attention sur la machine de développement

- Le port **5433** était déjà pris par un autre conteneur (`launchops-db`). PostgreSQL du projet utilise donc le port **5434**.
- Au démarrage, Docker Desktop relance un conteneur `open-webui` sur le port **3000**. Si ce conteneur tourne, `next dev` se rabattra sur le port 3001.
- Le dossier `archive/tmp/github-publish` contient un ancien clone git. Il a été déplacé tel quel.

3 Architecture globale

3.1 Vue d'ensemble

L'architecture reprend la section 3.1 du CDC. Le navigateur ne connaît qu'une seule adresse, celle de l'**API Gateway**. C'est la gateway qui vérifie l'identité de l'utilisateur, puis transmet la requête au microservice concerné.

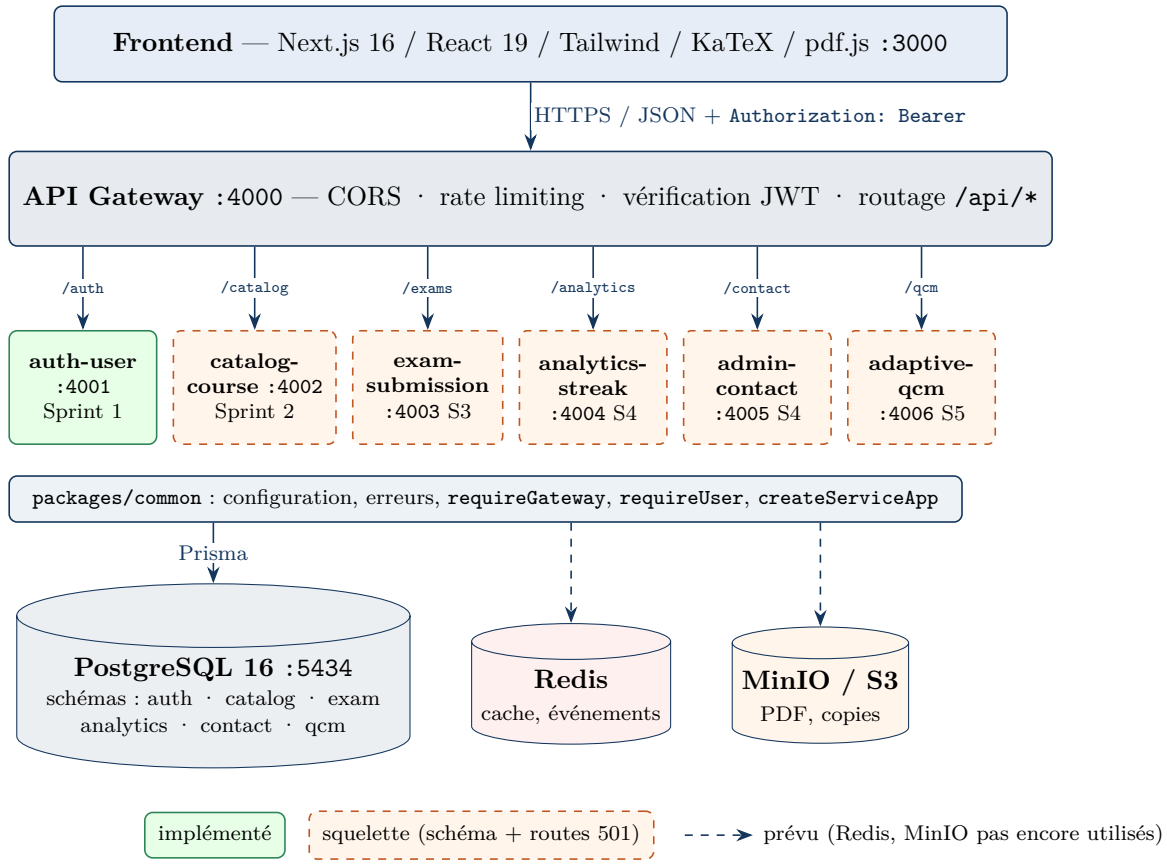


Figure 2 – Architecture microservices : en vert ce qui est implémenté, en orange les squelettes

3.2 Principes de conception

Un seul point d'entrée
Le frontend appelle uniquement `/api/...` sur la gateway. Les services ne sont jamais appelés directement par le navigateur.

JWT vérifié une seule fois
La gateway valide le jeton, puis transmet l'identité au service dans les en-têtes `x-user-id` et `x-user-role`. Les services n'ont pas besoin du secret JWT ; seul `auth-user-service` le connaît, car c'est lui qui signe les jetons.

Confiance gateway → service
Chaque requête transmise porte un secret partagé, `x-internal-secret`. Un service refuse (403) toute requête qui ne le présente pas : un client qui l'appellerait directement ne peut donc pas usurper une identité.

Une base par service
Il n'y a qu'un seul serveur PostgreSQL, mais chaque service a son propre schéma et son propre client Prisma (généré dans `src/generated/prisma`). Aucun service ne lit les tables d'un autre. Les références entre services, comme `userId`, sont de simples UUID sans clé étrangère.

Code commun factorisé
Le paquet `@cnc/common` évite la duplication entre services : chargement du `.env`, format des erreurs, contrôle d'accès et construction de l'application Express.

3.3 Cycle de vie d'une requête authentifiée

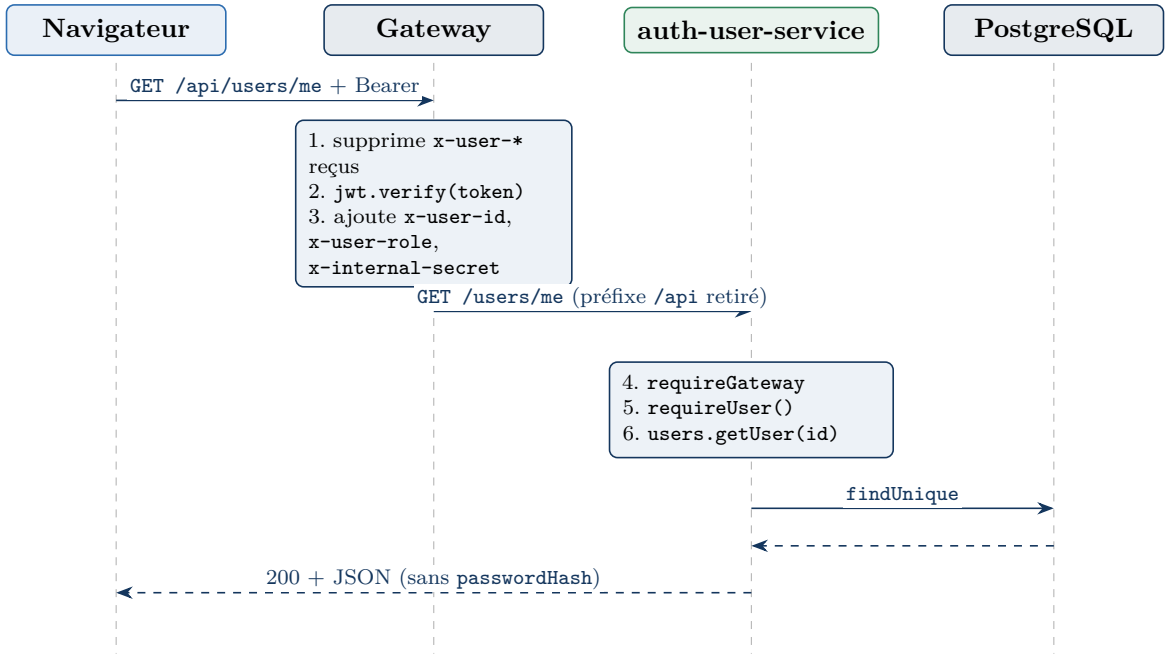


Figure 3 – Diagramme de séquence : lecture du profil de l'utilisateur connecté

4 Frontend : ce qui a été fait

4.1 Pile technique et volume

Bibliothèque	Rôle dans le projet
Next.js 16 (App Router), React 19, TypeScript	Structure des pages ; tous les écrans sont des composants client (`'use client'`)
Tailwind CSS v4, `tw-animate-css`	Styles ; mode sombre piloté par la classe `.dark` sur `<html>`
KaTeX 0.19	Rendu des formules (`MathFormula.tsx` appelle `katex.renderToString`)
`pdfjs-dist` 6.5	Liseuse PDF intégrée (`document-studio/`) : pages, recherche, signets, mode examen
`framer-motion`	Animations (cartes de dossiers, lecteur d'épreuve)
`lucide-react`, `@phosphor-icons/react`	Icônes

Table 2 – Bibliothèques principales du frontend

Dossier	Fichiers	Lignes
`app/`	9	2 313
`components/`	42	25 471
`lib/` (dont 2 manifestes JSON)	8	9 326
`scripts/`	6	663
Total	65	**37 773**

Ressources	Volume
PDF d'annales	559 (568 Mo)
PDF de problèmes	20 (1,9 Mo)
Épreuves référencées	437
Chapitres de cours	26

Table 3 – Volume de code et de contenu du frontend

4.2 Routes et navigation

Route	Fichier	Contenu
`/`	`app/page.tsx`	Application étudiant : 6 onglets (Dashboard, Cours, Annales, Problèmes, QCM, Contact)
`/login`	`app/login/page.tsx`	`LoginPage`, puis redirection vers `/admin` ou `/` selon le rôle
`/signup`	`app/signup/page.tsx`	`SignUpPage` (nom, e-mail, centre, année, filière)
`/admin`	`app/admin/page.tsx`	`AdminBackOffice` : messages, catalogue, annales, QCM
`/pochettes`	`app/pochettes/page.tsx`	Espace personnel en dossiers, éditeur de pages (LaTeX, PDF)
`/parametres`	`app/parametres/page.tsx`	Profil, cursus, sécurité, notifications

Table 4 – Routes de l'application

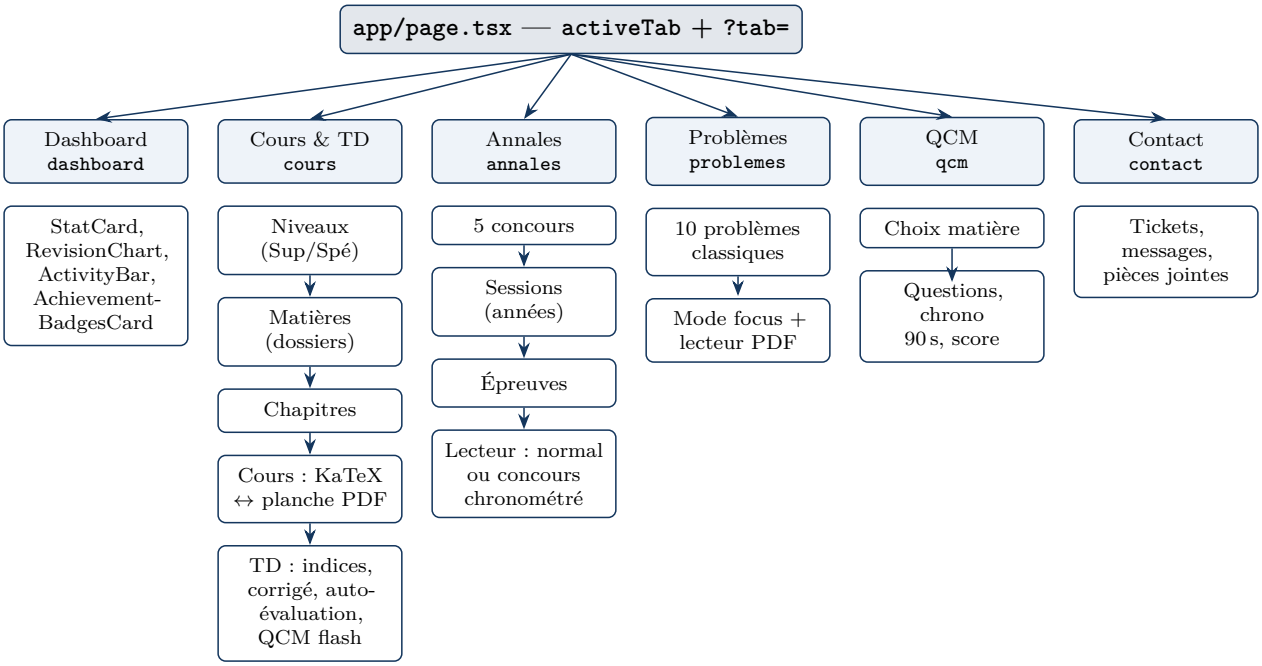


Figure 4 – Arbre de navigation de l'espace étudiant

Les onglets Cours, Annales et QCM sont tous les trois rendus par le même composant, `CourseExerciseQcmDoubleMode.tsx` (5 276 lignes), avec une valeur différente de la prop `initialSection`.

Logique de navigation. `app/page.tsx` garde l'onglet actif dans `activeTab`. `navigateToTab()` met à jour cet état et écrit `?tab=<id>` dans l'URL avec `history.pushState`. Au chargement et sur `popstate`, `syncTabFromUrl()` relit le paramètre, en n'acceptant que les onglets d'une liste blanche. Les boutons Précédent et Suivant du navigateur fonctionnent donc sans rechargement.

Session (simulée). `lib/auth.ts` enregistre un objet `UserProfile` dans `localStorage` (clé `learning_cnc_user_session`). Les pages le lisent avec `getStoredUser() || DEMO_STUDENT`. Elles se synchronisent grâce à l'événement `nexora-user-updated`, émis par la page des paramètres.

Thème. `ThemeToggle` lit `localStorage['nexora-theme']`, à défaut `prefers-color-scheme`, puis bascule la classe `dark` sur `<html>`. Le fond animé WebGL (`SilkBackground`) observe cette classe avec un `MutationObserver`.

4.3 Écrans réalisés

4.3.1 Authentification (M1)

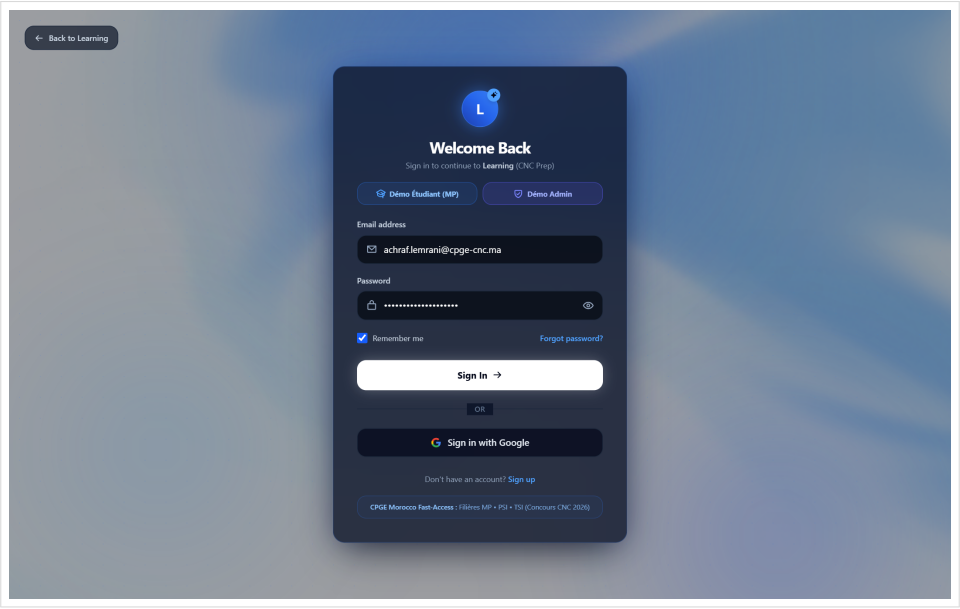


Figure 5 – Connexion : comptes de démonstration étudiant et administrateur

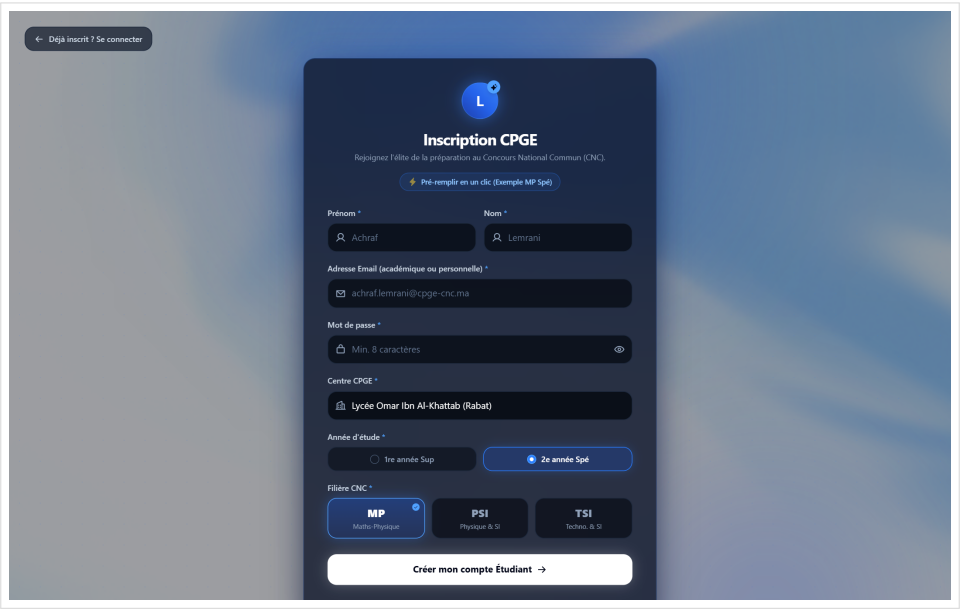


Figure 6 – Inscription : prénom, nom, e-mail, centre CPGE, année et filière (UC01)

Logique : `LoginPage` renvoie `DEMO_ADMIN` si l'e-mail est celui de l'administrateur, sinon `DEMO_STUDENT`, puis appelle `saveUserSession()`. **Le mot de passe n'est pas vérifié** : c'est précisément le rôle du backend (section 5.4).

4.3.2 Dashboard (M5)

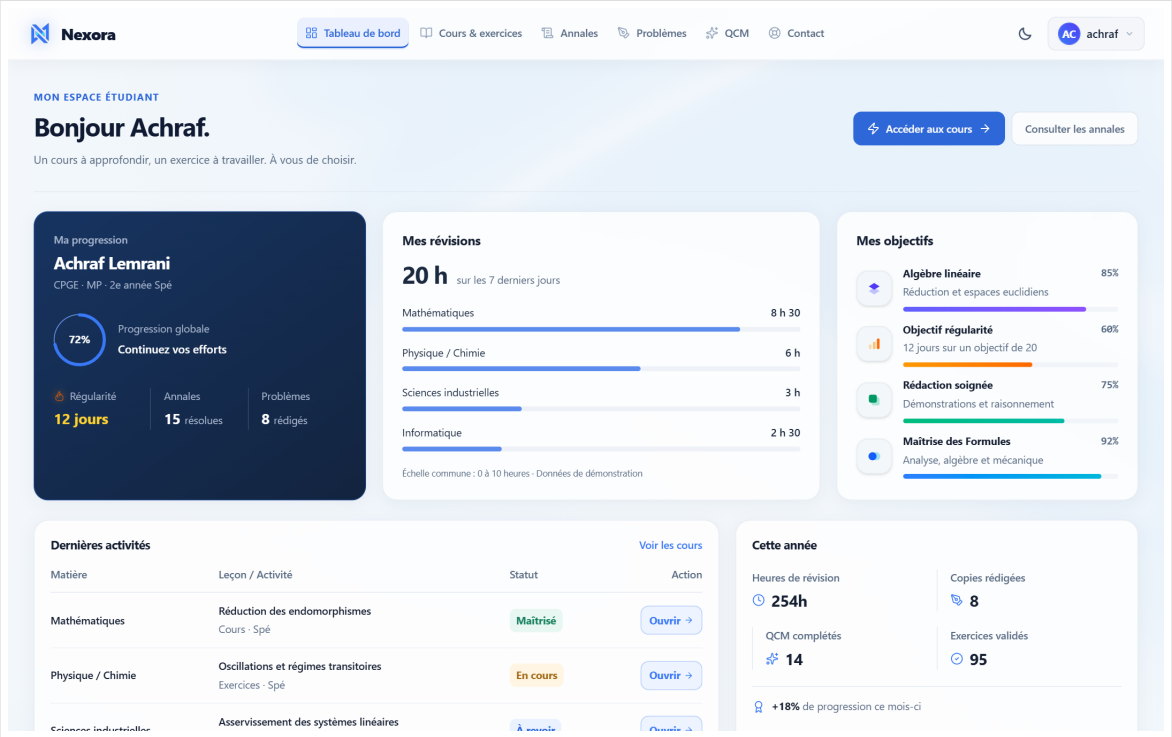


Figure 7 – Dashboard étudiant : progression, temps de révision par matière, objectifs, activité

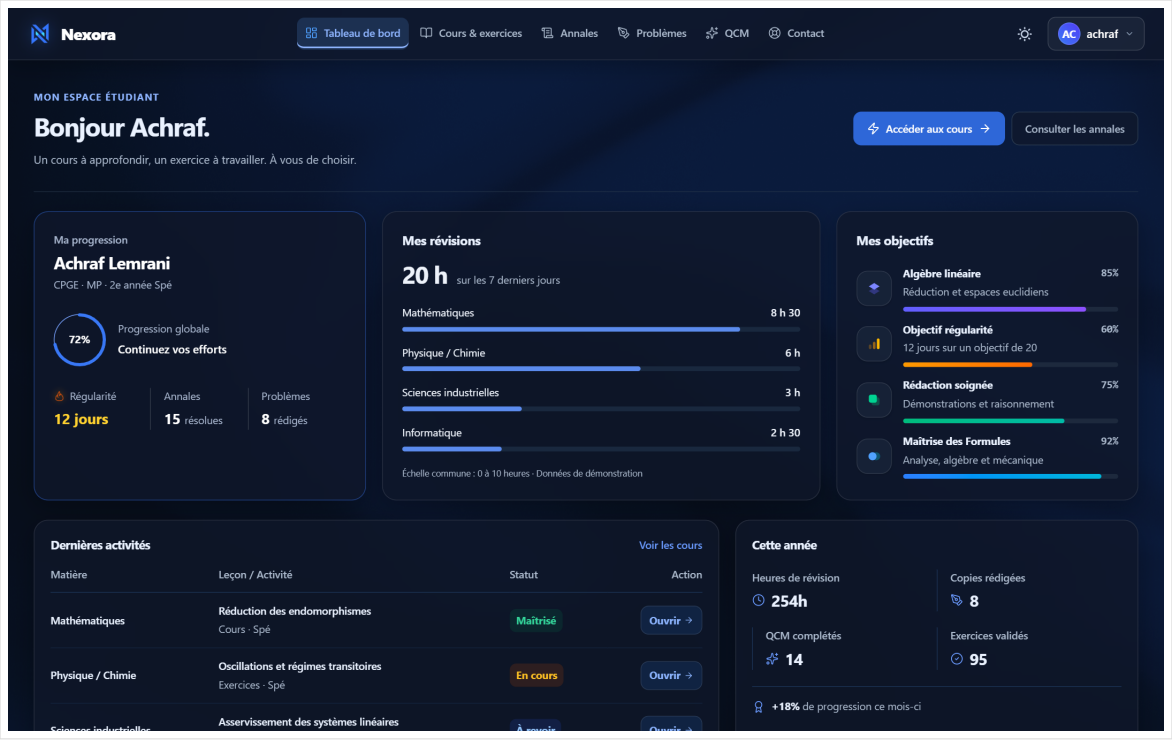


Figure 8 – Le même écran en mode sombre (exigence non fonctionnelle 7.1)

Les valeurs affichées (72 %, 12 jours, 15 annales, 254 h…) sont **écrites en dur**. La heatmap (`ActivityBar`) est générée par une formule pseudo-aléatoire. Ces données viendront de `analytics-streak-service`.

4.3.3 Cours et exercices en double mode (M2)

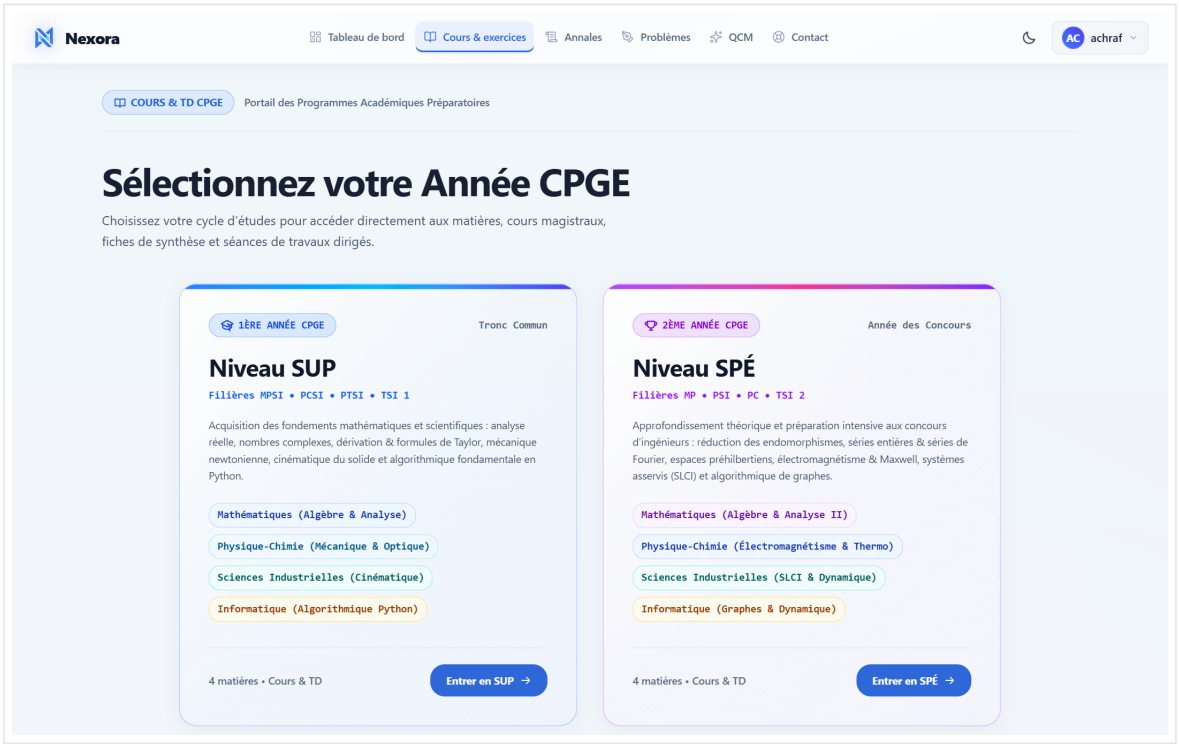


Figure 9 – Choix du niveau : accès libre Sup et Spé

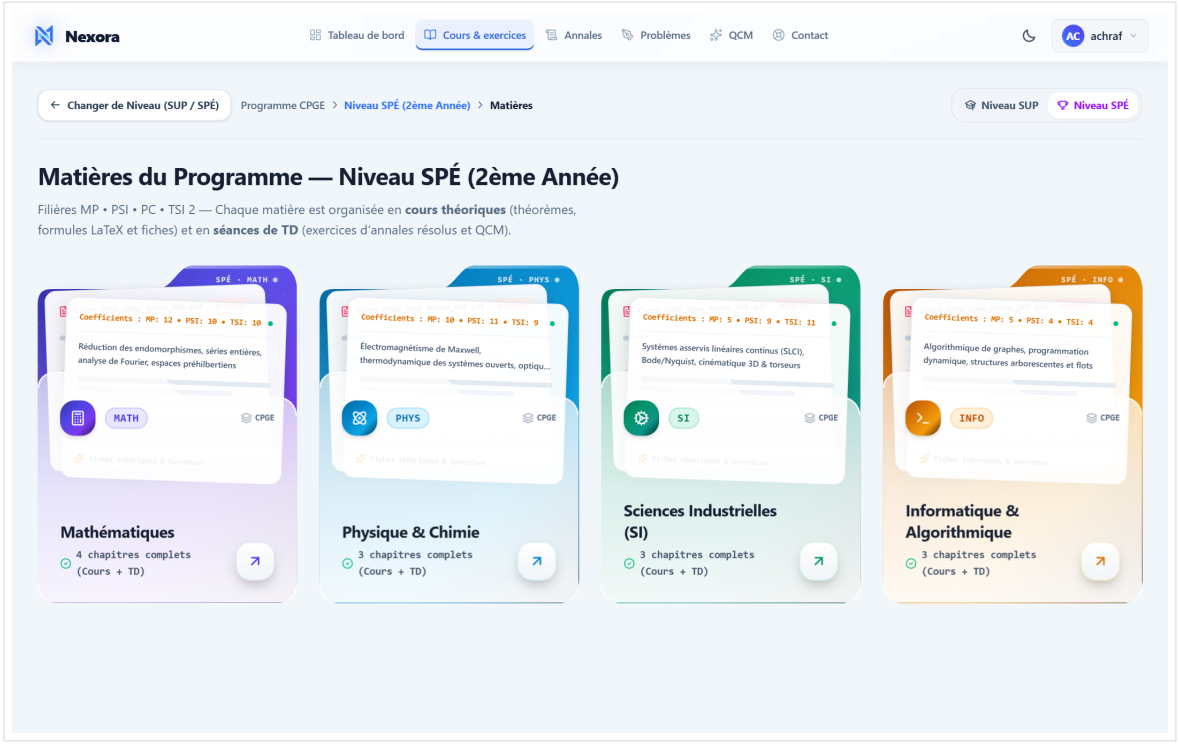


Figure 10 – Choix de la matière (cartes « dossiers »)

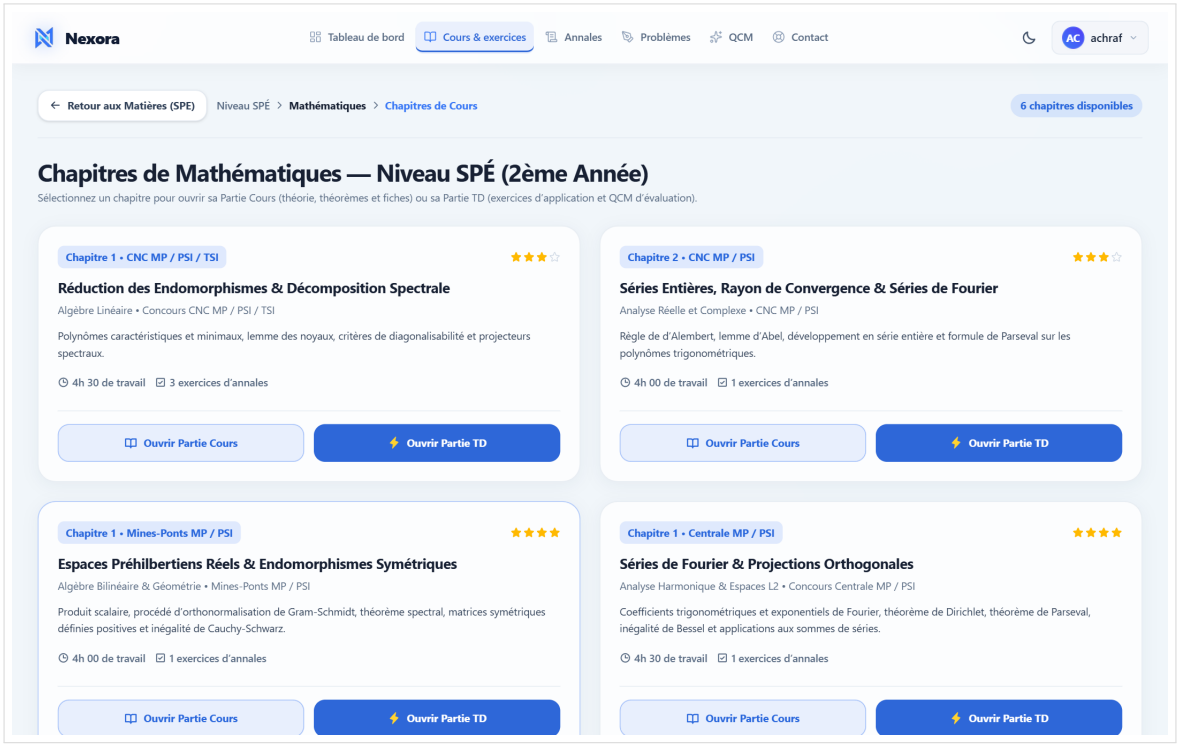


Figure 11 – Liste des chapitres d'une matière, avec accès à la partie Cours ou TD

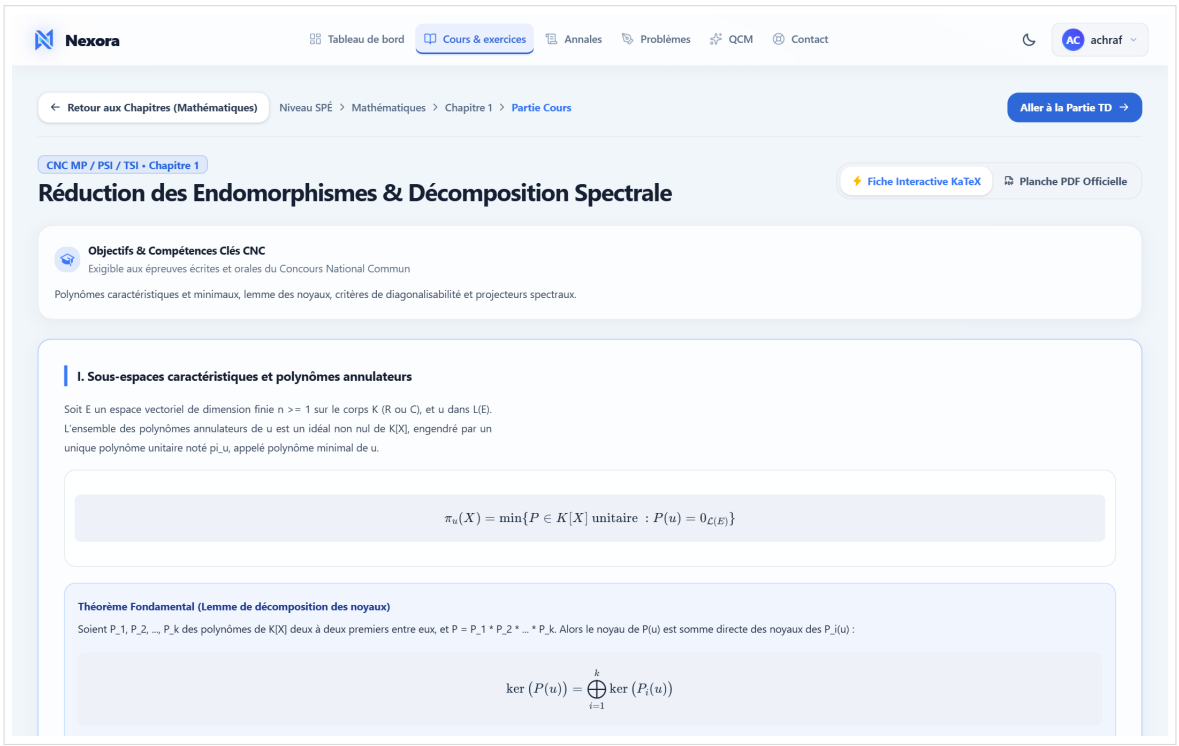


Figure 12 – Partie Cours en mode « Fiche interactive KaTeX » : théorie, théorèmes, formules

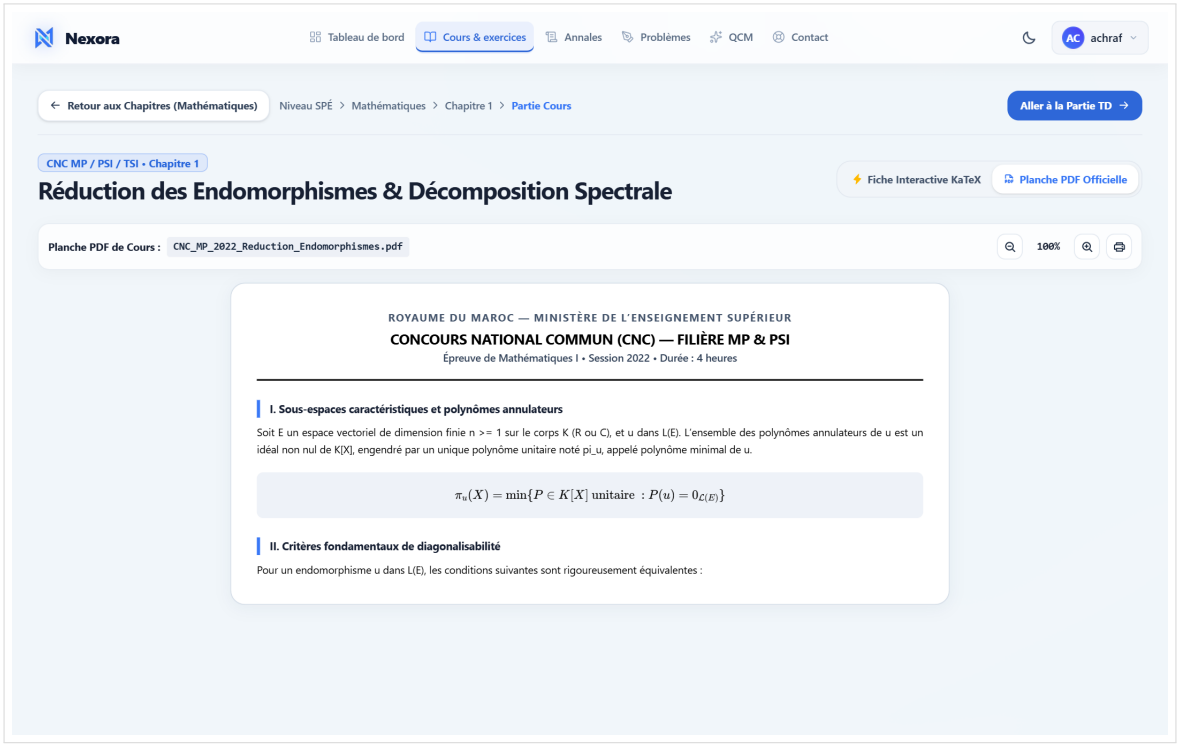


Figure 13 – Partie Cours en mode « Planche PDF » (rendu HTML façon document officiel)

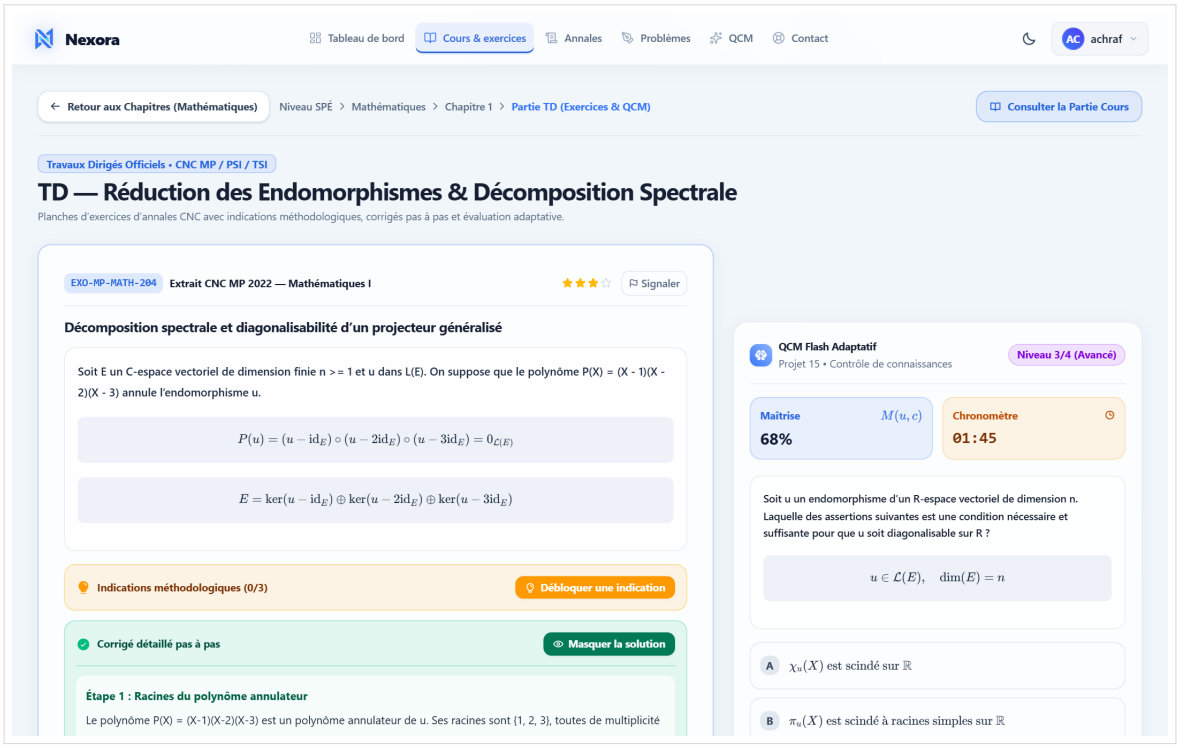


Figure 14 – Partie TD : exercice référencé (`EXO-MP-MATH-204`), indications progressives, corrigé pas à pas et QCM flash

Logique : toutes les données (26 chapitres, 28 exercices, 26 questions) sont dans le tableau `COURSES_DATABASE` de `CourseExerciseQcmDoubleMode.tsx`. Chaque exercice a le type suivant :

```
interface CourseExercise {
  id: string; refCode: string; source: string; difficultyStars: number
  title: string; statement: string; statementMath?: string[]
```

```
  hints: string[]                                    // indices progressifs
  solutionSteps: { stepTitle: string; content: string; math?: string }[]
}
```

Le composant mémorise `unlockedHints[exoId]`, le nombre d'indices déjà dévoilés, et `revealedSolutions[exoId]`. L'auto-évaluation (Maîtrisé, Avec aide, À revoir) est stockée dans `exerciseEvaluations`, **uniquement en mémoire**. Ce type correspond presque exactement au modèle `Exercise` du backend (`statementLatex`, `hintsLatex[]`, `solutionLatex`).

4.3.4 Annales CNC et simulateur d'épreuve (M3)

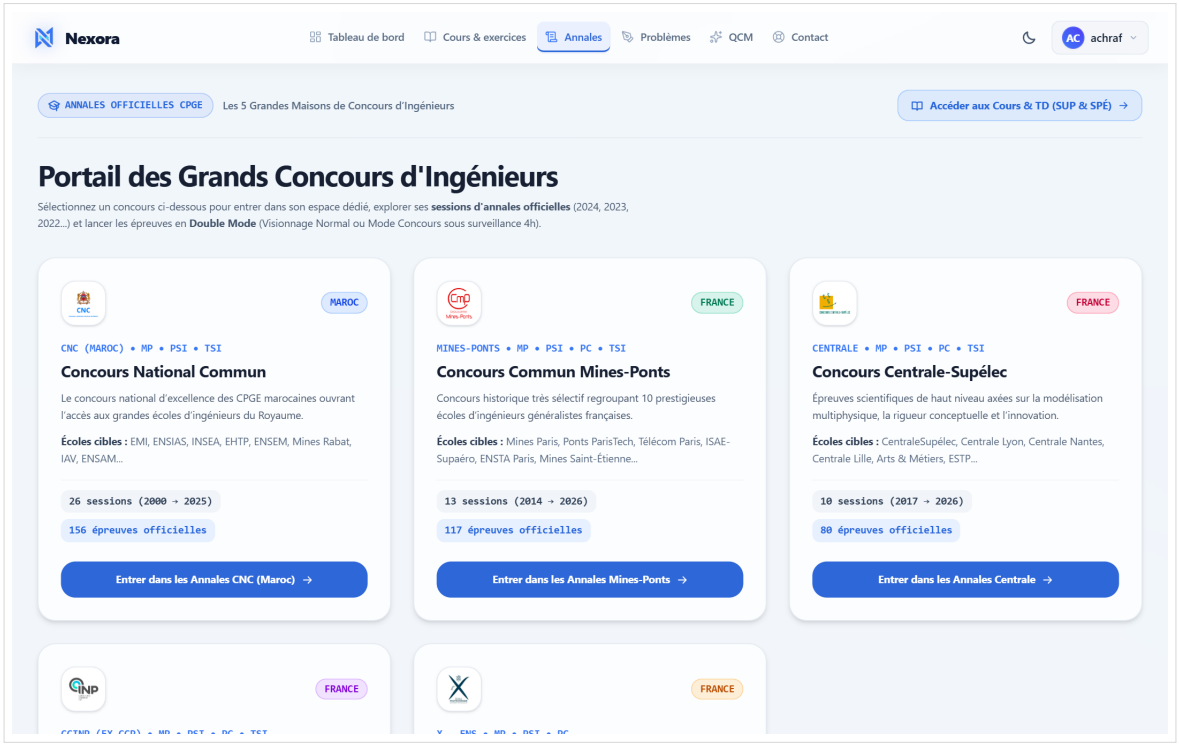


Figure 15 – Portail des concours : 437 épreuves référencées, dont 156 CNC avec de vrais PDF

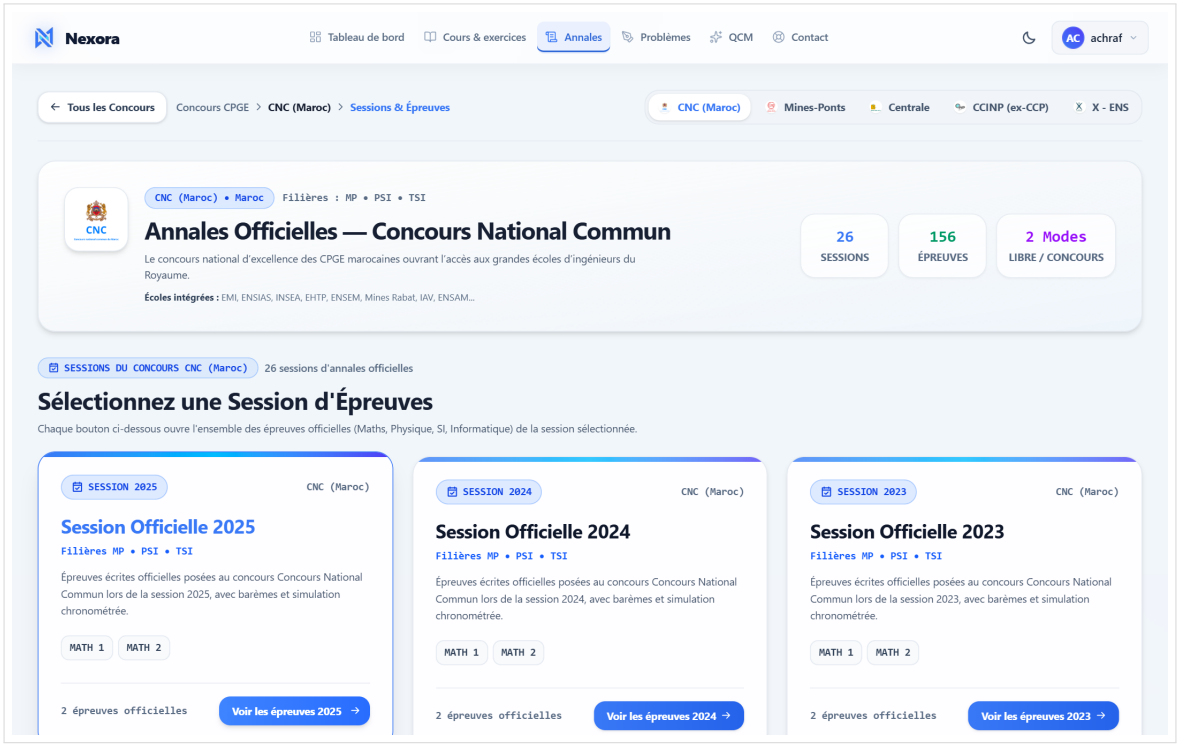


Figure 16 – Sessions d'annales CNC (2000 à 2025)

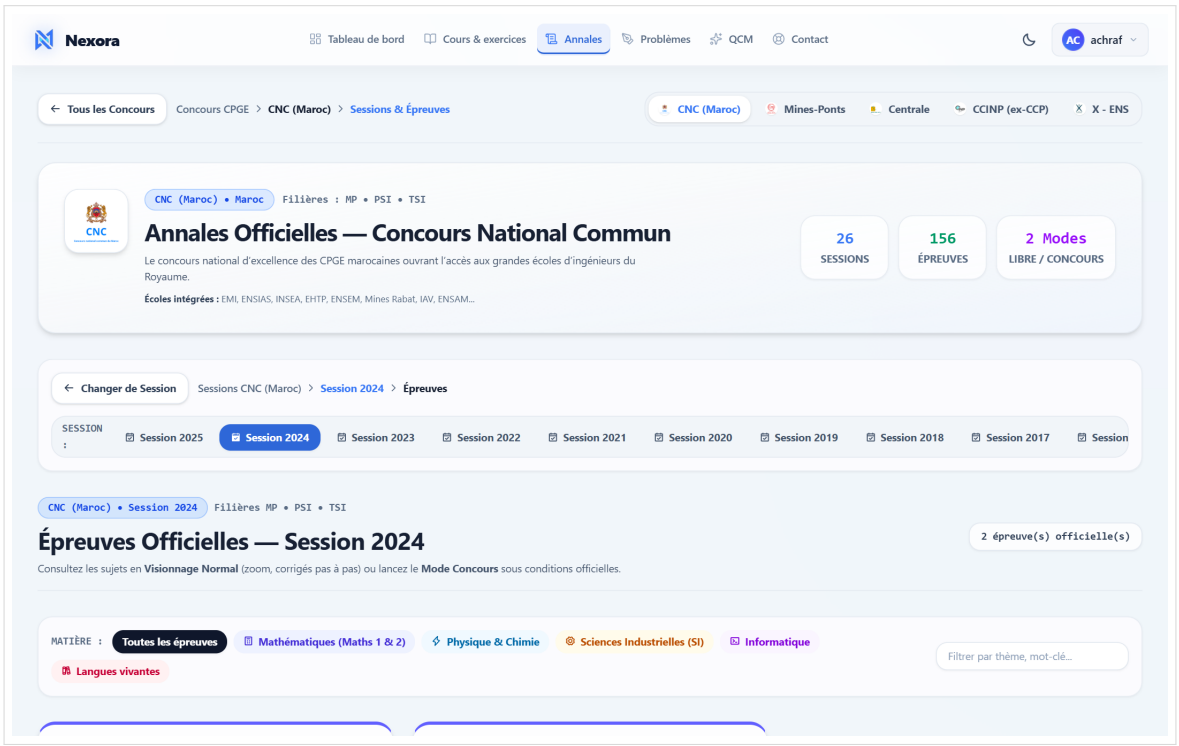


Figure 17 – Épreuves d'une session, filtres par matière, visionnage normal ou mode concours

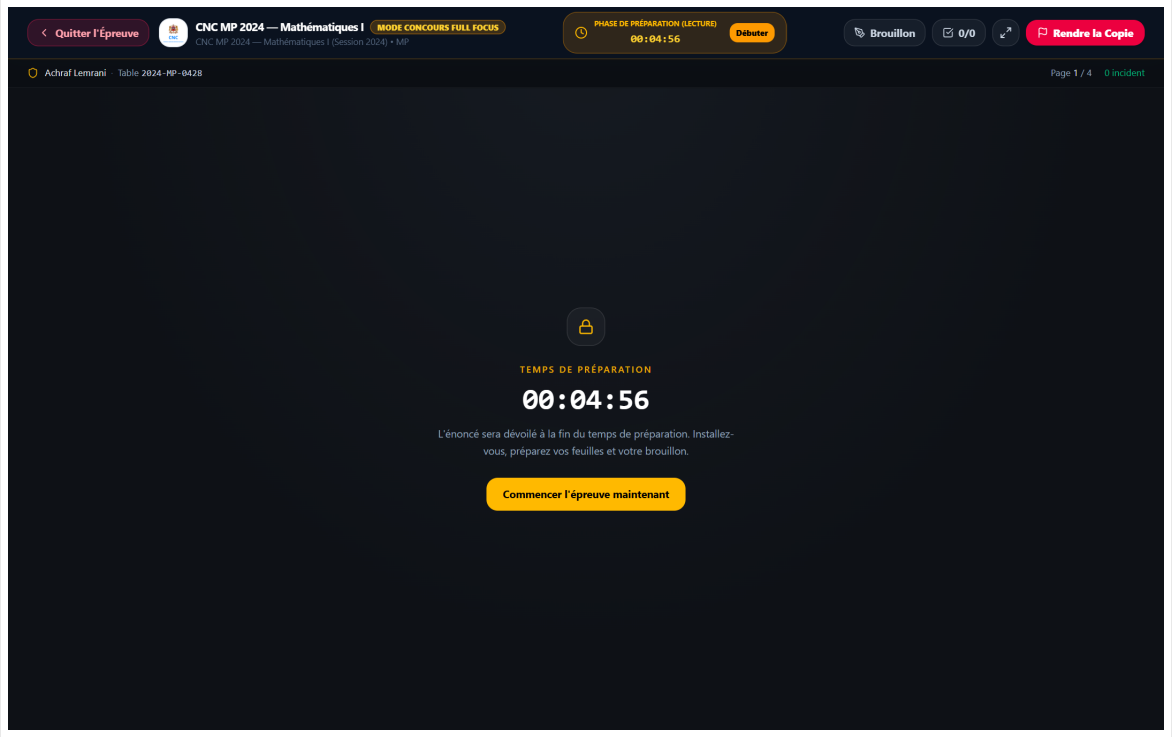
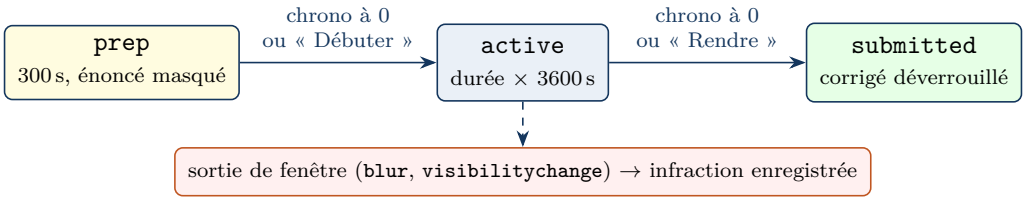


Figure 18 – Mode concours : phase de préparation, chronomètre, brouillon, remise de copie

Logique du chronomètre (`ConcoursPdfExamViewer.tsx`) : l'épreuve passe par trois phases.



Un `setInterval` d'une seconde décrémente le temps restant. Les annales sont décrites par deux manifestes JSON, générés par `scripts/download-*-annales.mjs`, qui pointent vers `public/annales/...`.

4.3.5 Problèmes, QCM et contact

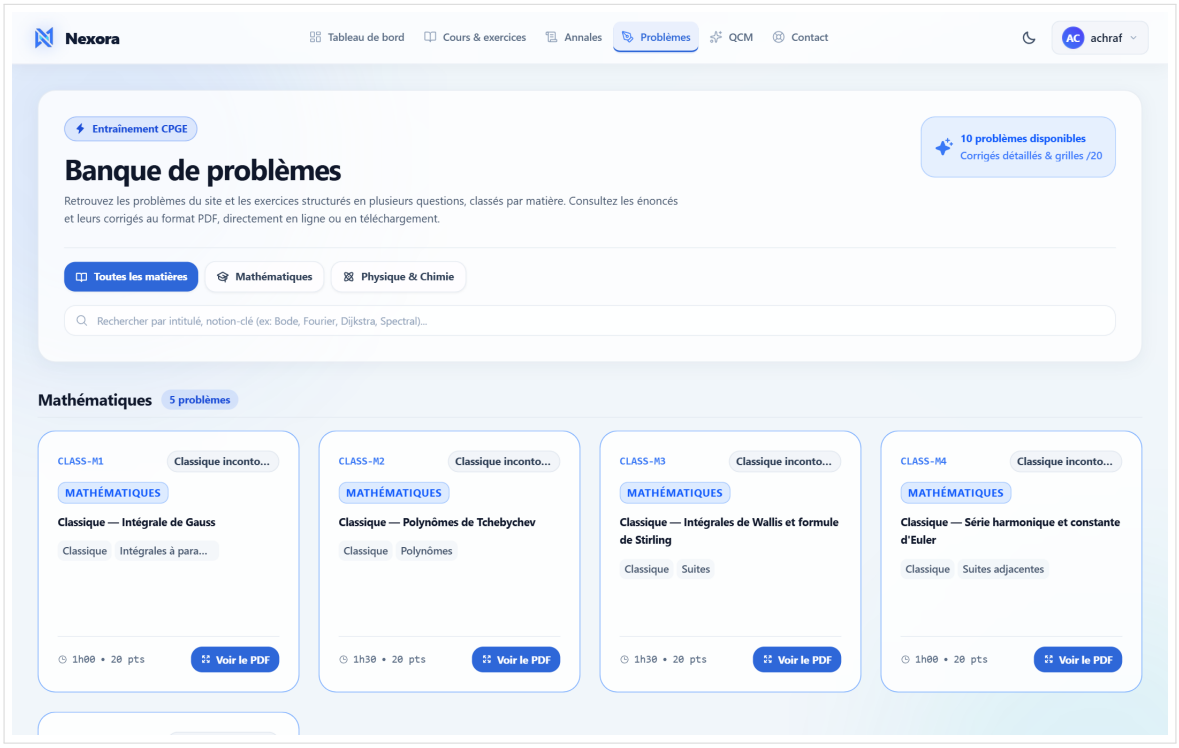


Figure 19 – Banque de problèmes classiques (énoncé et corrigé PDF générés par `build-problem-pdfs.cjs`)

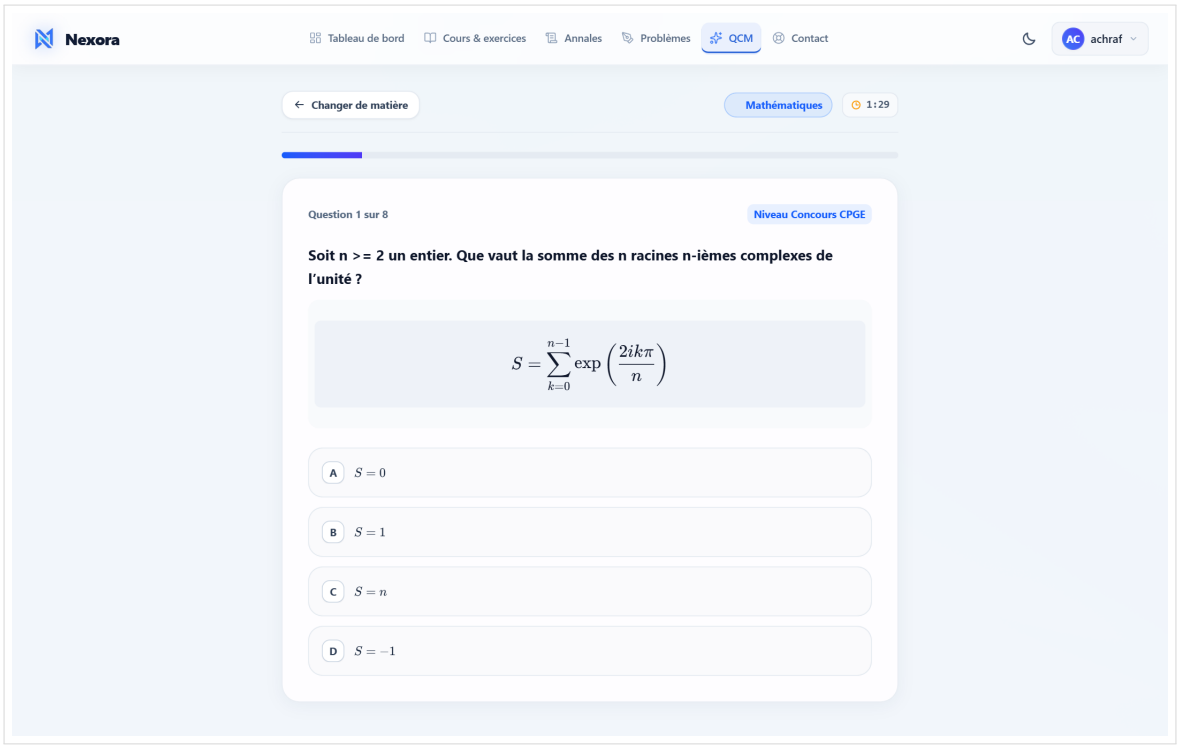


Figure 20 – QCM par matière : question, choix rendus en KaTeX, chronomètre de 90 s

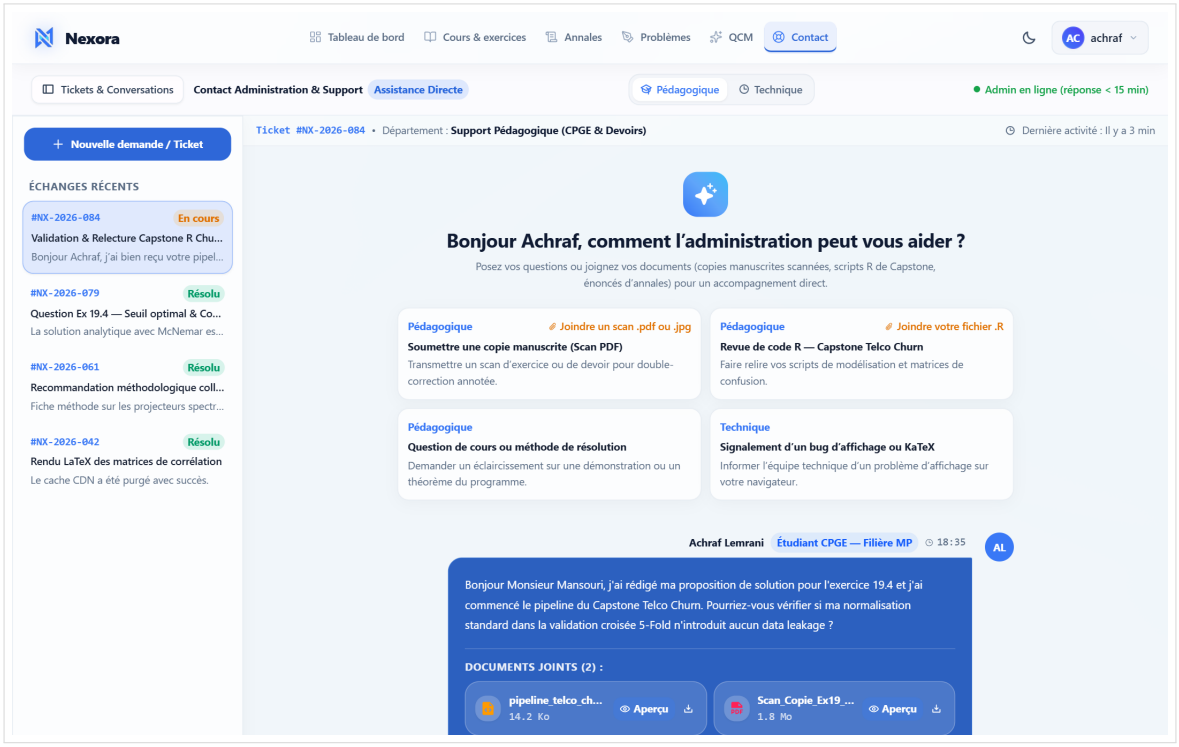


Figure 21 – Contact admin : tickets, messagerie, pièces jointes (M6)

4.3.6 Back-office et espace personnel

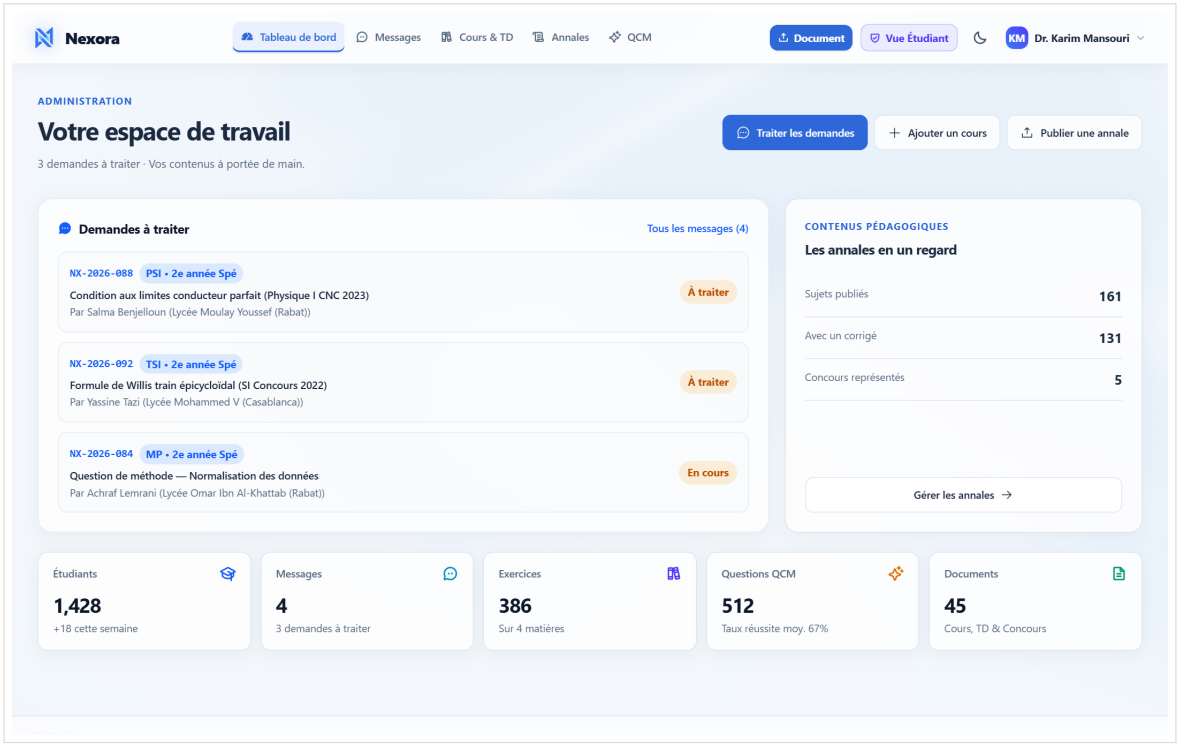


Figure 22 – Back-office administrateur : demandes à traiter, annales, statistiques

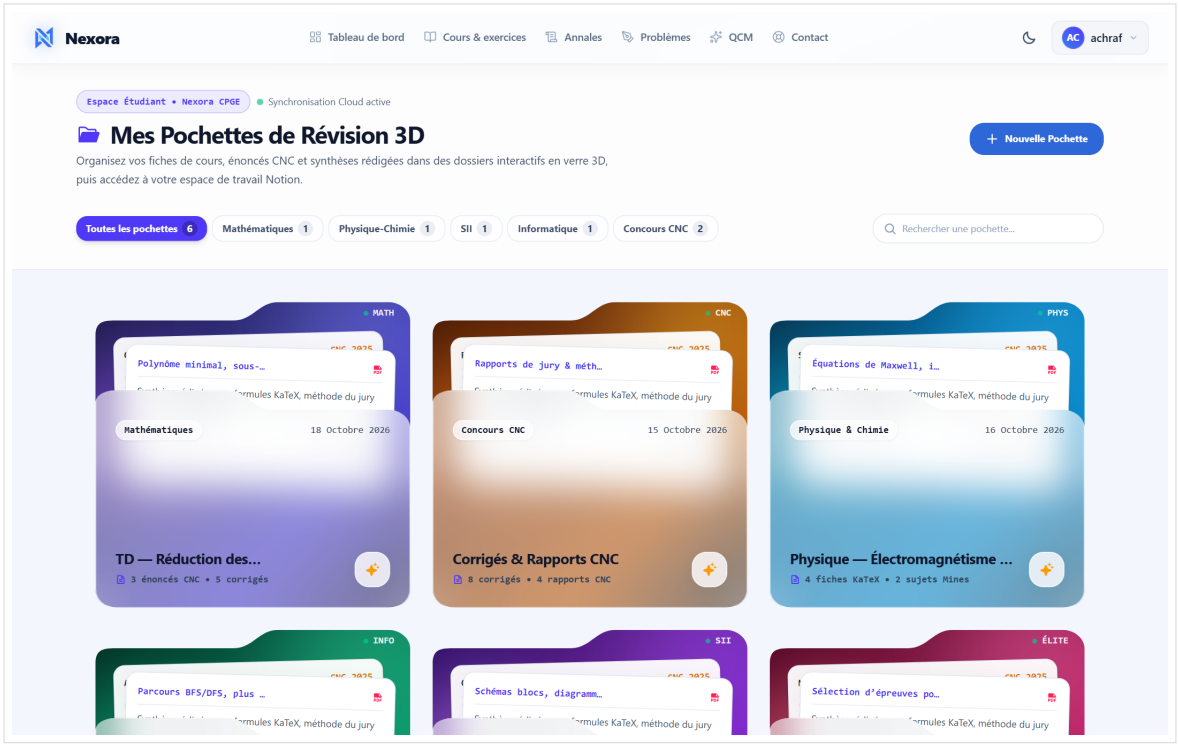


Figure 23 – Pochettes : espace personnel organisé en dossiers

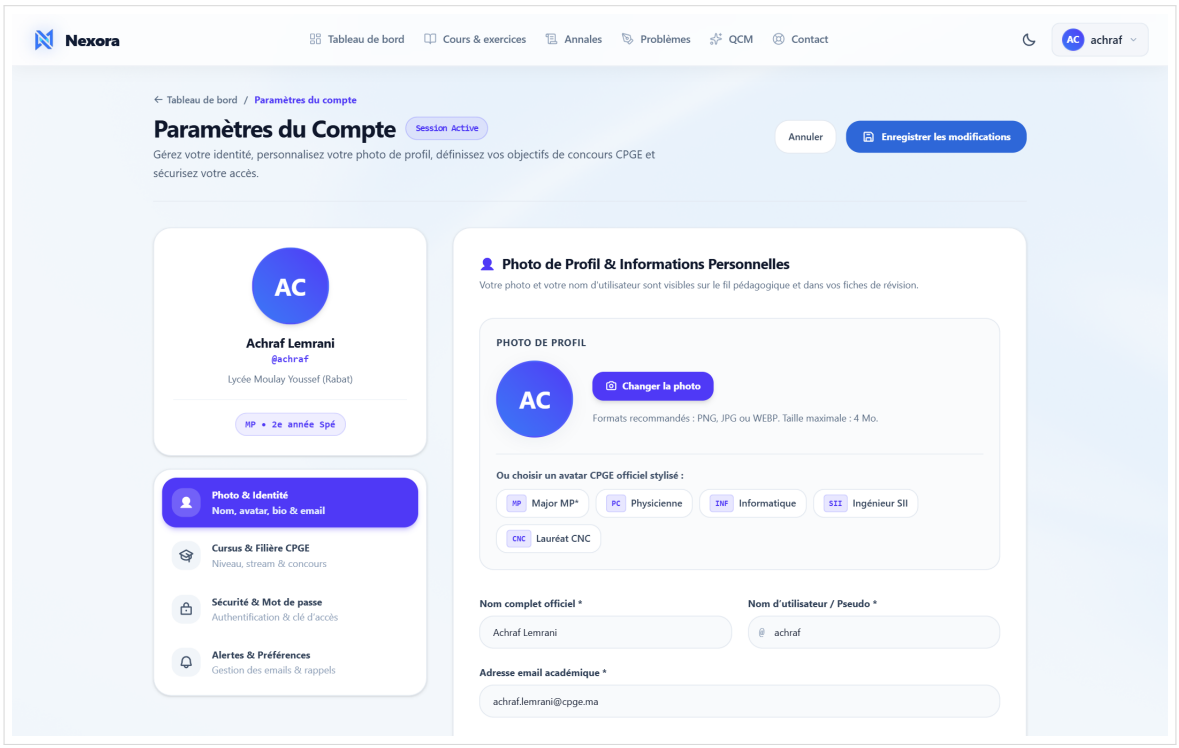


Figure 24 – Paramètres : profil, cursus, sécurité, notifications

4.4 Bilan du frontend

Points forts

L'interface est complète et cohérente avec les maquettes du CDC : double mode d'exercices, rendu KaTeX, liseuse PDF avec mode examen, mode sombre, mise en page responsive, 559 PDF d'annales réellement intégrés, et une navigation qui conserve l'onglet dans l'URL.

Limites : c'est un prototype

- **Aucun appel réseau** (aucun `fetch`) : toutes les données sont en dur ou dans `localStorage`, et rien n'est conservé côté serveur.
- **Pas de vraie sécurité** : le mot de passe n'est pas vérifié, la route `/admin` est accessible sans contrôle de rôle, et `clearUserSession()` n'est jamais appelée à la déconnexion.
- **Composants très volumineux** : `CourseExerciseQcmDoubleMode.tsx` fait 5 276 lignes et `AdminBackOffice.tsx` 3 726 lignes. Ils mélangent données et affichage.
- **Code orphelin** : `StudentDrawer`, `PersonalWorkspaceModal` et `glass-folder-cards` ne sont rendus nulle part ; l'onglet `fiches` est inaccessible.
- Le chronomètre du QCM flash est figé sur « 01 :45 » ; la répétition espacée n'est pas reliée à l'interface.
- `next.config.mjs` contient `ignoreBuildErrors: true` : les erreurs TypeScript sont masquées au build.

5 Backend : ce qui a été fait

5.1 Organisation en monorepo

Le backend est un **monorepo npm workspaces**. Un seul `npm install` à la racine de `backend/` installe les dépendances de la gateway, du paquet commun et des six services. Chaque service reste pourtant un paquet indépendant, avec son `package.json`, son schéma Prisma et ses scripts.

Listing 2 – Arborescence du backend (1 336 lignes de TypeScript et Prisma écrites à la main)

```
backend/
+-- package.json            workspaces + scripts globaux (dev, typecheck, test, db:*)
+-- tsconfig.base.json      options TypeScript communes (strict, ES2023, Bundler)
+-- compose.yaml            postgres:16 (5434), redis:7 (6379), minio (9000/9001)
+-- .env.example            toutes les variables, une URL de base par service
+-- packages/common/src/
|   +-- env.ts              charge backend/.env, requireEnv(), portEnv()
|   +-- errors.ts           HttpError, notFoundHandler, errorHandler (Zod -> 400)
|   +-- auth-context.ts     en-têtes x-user-*, requireGateway(), requireUser(), currentUser()
|   +-- service-app.ts      createServiceApp() : /health + sécurité + JSON + erreurs
|   +-- not-implemented.ts  notImplemented('Sprint N') -> 501
+-- gateway/src/
|   +-- index.ts            CORS, rate limit, /api/health, proxys
|   +-- authenticate.ts     vérification JWT + injection d'identité
|   +-- routes.ts           table préfixe -> service, routes publiques
+-- services/
    +-- auth-user-service/      IMPLÉMENTÉ (Sprint 1)
    |   +-- prisma/schema.prisma, prisma/seed.ts
    |   +-- src/ index, config, db, schemas, tokens, user-mapper,
    |            auth.service, users.service, routes, schemas.test
    +-- catalog-course-service/   prisma/schema.prisma + src/index.ts (routes 501)
    +-- exam-submission-service/  idem
    +-- analytics-streak-service/ idem + src/streak.ts (+ tests)
```

```
      +-- admin-contact-service/    idem
      +-- adaptive-qcm-service/     idem + src/adaptive-engine.ts (+ tests)
```

Commande (dans `backend/`)	Effet
`npm run infra:up`	Démarre PostgreSQL, Redis et MinIO avec Docker Compose
`npm run db:generate`	Génère les 6 clients Prisma (`src/generated/prisma` de chaque service)
`npm run db:push`	Crée ou met à jour les 6 schémas PostgreSQL
`npm run db:seed`	Crée les comptes de démonstration (admin et étudiant)
`npm run dev`	Lance les 7 processus en parallèle (`concurrently` + `tsx watch`)
`npm run typecheck`	`tsc` strict sur chaque workspace
`npm test`	Tests unitaires (`node:test` via `tsx`)

Table 5 – Scripts du monorepo

Choix techniques

— **Express 5** plutôt que NestJS (le CDC accepte les deux). Express est plus simple à relire, et sa version 5 transmet automatiquement les erreurs des fonctions `async` au gestionnaire d'erreurs, sans `try/catch` dans chaque route.

— **tsx** exécute le TypeScript directement, sans étape de compilation en développement. `tsc --noEmit` sert uniquement à la vérification de types.

— **Zod** valide toutes les entrées. Une donnée invalide produit une erreur `VALIDATION_ERROR` (400) qui détaille chaque champ fautif.

— **Un fichier `.env` unique** dans `backend/`, chargé par `@cnc/common` avec `process.loadEnvFile` (fonction native de Node). Chaque service y lit sa propre URL de base (`AUTH_DATABASE_URL`, `CATALOG_DATABASE_URL`, etc.).

5.2 Le paquet partagé `@cnc/common`

Tous les services sont construits de la même façon, grâce à `createServiceApp`. Pour relire un service, il suffit donc de lire son fichier `routes` : le reste est commun.

Listing 3 – `packages/common/src/service-app.ts` (extrait)

```
export function createServiceApp({ name, internalSecret, router }: ServiceOptions): Express {
  const app = express()
  app.disable('x-powered-by')

  app.get('/health', (_req, res) => {                // public : supervision
    res.json({ service: name, status: 'ok', time: new Date().toISOString() })
  })

  app.use(requireGateway(internalSecret))            // 403 si l'appel ne vient pas de la
    gateway
  app.use(express.json({ limit: '1mb' }))
  app.use(router)                                    // routes métier du service

  app.use(notFoundHandler)                           // 404 JSON
  app.use(errorHandler)                              // HttpError, ZodError, 500
  return app
}
```

Listing 4 – `auth-context.ts` : contrôle d'accès dans les services

```
export function requireUser(role?: Role): RequestHandler {
  return (req, _res, next) => {
```

```
    const user = getAuthUser(req)                    // lit x-user-id / x-user-role
    if (!user) return next(new HttpError(401, 'Authentification requise', 'UNAUTHORIZED'))
    if (role && user.role !== role) return next(new HttpError(403, 'Droits insuffisants', '
    FORBIDDEN'))
    next()
  }
}
```

Toutes les erreurs ont le même format JSON, quel que soit le service. Le frontend n'aura donc qu'un seul cas à gérer :

```
{ "error": "EMAIL_TAKEN", "message": "Un compte existe deja avec cet e-mail" }
{ "error": "VALIDATION_ERROR", "message": "Donnees invalides",
  "details": [ { "path": "filiere", "message": "Invalid option: ..." } ] }
```

5.3 L'API Gateway

5.3.1 Chaîne de traitement d'une requête

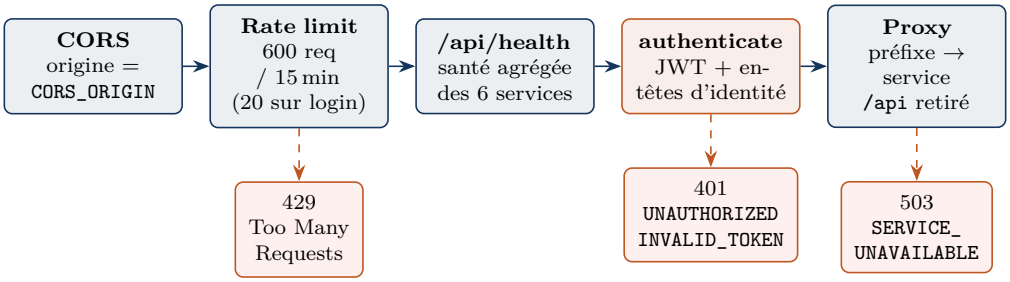


Figure 25 – Middlewares de la gateway, dans l'ordre d'exécution

La gateway **ne lit pas le corps** des requêtes (pas de `express.json()`). Le flux est transmis tel quel au service, ce qui évite de devoir le reconstruire dans le proxy.

5.3.2 Table de routage

Préfixe public	Service	Cible	Exemple réécrit
`/api/auth`	auth-user-service	`:4001`	`/api/auth/login` → `/auth/login`
`/api/users`	auth-user-service	`:4001`	`/api/users/me` → `/users/me`
`/api/catalog`	catalog-course-service	`:4002`	`/catalog/chapters`
`/api/exams`	exam-submission-service	`:4003`	`/exams/:id/sessions`
`/api/analytics`	analytics-streak-service	`:4004`	`/analytics/me/streak`
`/api/contact`	admin-contact-service	`:4005`	`/contact/tickets`
`/api/qcm`	adaptive-qcm-service	`:4006`	`/qcm/sessions`

Table 6 – Routage de la gateway (`gateway/src/routes.ts`)

Routes publiques (sans jeton) : `POST /api/auth/register`, `/login`, `/refresh`, `/logout` et `GET /api/health`. Toutes les autres routes exigent un access token valide.

Listing 5 – `gateway/src/authenticate.ts` : le cœur de la sécurité

```
export function authenticate(accessSecret: string, internalSecret: string): RequestHandler {
  return (req, _res, next) => {
    delete req.headers[HEADER_USER_ID]               // anti-usurpation : on ignore
```

```
    delete req.headers[HEADER_USER_ROLE]              // ce que le client prétend être
    req.headers[HEADER_INTERNAL_SECRET] = internalSecret

    const header = req.header('authorization')
    const token = header?.startsWith('Bearer ') ? header.slice(7) : null
    if (token) {
      try {
        const payload = jwt.verify(token, accessSecret) as AccessTokenPayload
        req.headers[HEADER_USER_ID] = payload.sub
        req.headers[HEADER_USER_ROLE] = payload.role
      } catch {
        return next(new HttpError(401, 'Access token invalide ou expire', 'INVALID_TOKEN'))
      }
    }
    if (!token && !isPublicRoute(req.method, req.path)) {
      return next(new HttpError(401, 'Authentification requise', 'UNAUTHORIZED'))
    }
    next()
  }
}
```

5.4 auth-user-service (Sprint 1, implémenté)

5.4.1 Découpage en couches

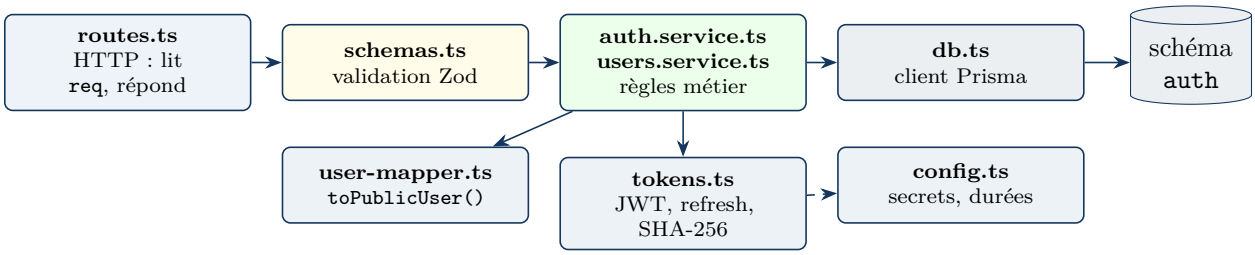


Figure 26 – Couches du service : chaque fichier a une seule responsabilité

5.4.2 Modèle de données (schéma `auth`)

Le modèle traduit la hiérarchie `User` / `Student` / `Administrator` du diagramme de classes du CDC (section 4.2). Comme Prisma ne gère pas l'héritage, on utilise une **table par spécialisation**, chacune en relation 1–1 avec `users`.

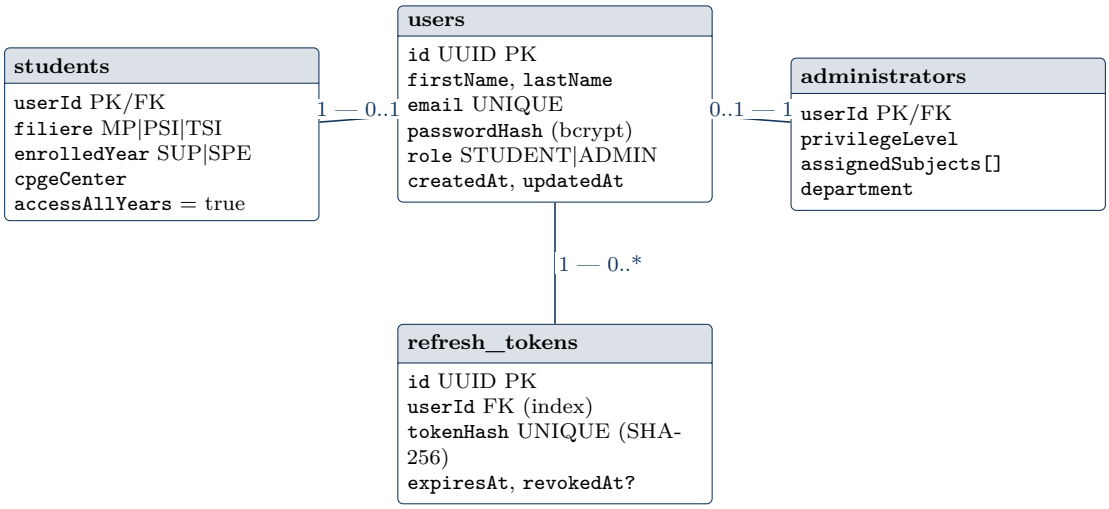


Figure 27 – Schéma `auth` (suppression en cascade depuis `users`)

5.4.3 Endpoints

Méthode	Route (gateway)	Accès	Corps / réponse
POST	`/api/auth/register`	public	`firstName, lastName, email, password, cpgeCenter, enrolledYear, filiere` → 201 `{user, accessToken, refreshToken}`
POST	`/api/auth/login`	public	`email, password` → 200 `{user, accessToken, refreshToken}`
POST	`/api/auth/refresh`	public	`refreshToken` → nouvelle paire (rotation)
POST	`/api/auth/logout`	public	`refreshToken` → 204 (jeton révoqué)
GET	`/api/users/me`	connecté	profil complet (jamais `passwordHash`)
PATCH	`/api/users/me`	connecté	nom, prénom ; centre, année, filière (étudiant)
GET	`/api/users?page=&pageSize=`	ADMIN	liste paginée `{total, page, items}`

Table 7 – API de `auth-user-service`

5.4.4 Stratégie des jetons

	Access token	Refresh token
Format	JWT signé HS256, contenu `{sub, role}`	48 octets aléatoires (`base64url`), opaque
Durée	15 minutes (`ACCESS_TOKEN_TTL`)	30 jours (`REFRESH_TOKEN_TTL_DAYS`)
Stockage serveur	aucun (vérifié par signature)	seulement son **empreinte SHA-256** en base
Vérifié par	la gateway, à chaque requête	`auth-user-service`, sur `/auth/refresh`
Révocation	expiration naturelle	`revokedAt` (déconnexion, rotation, vol détecté)

Table 8 – Les deux jetons définis par le CDC (section 2.1)

5.4.5 Rotation des refresh tokens et détection de vol

Chaque appel à `/auth/refresh` **consomme** le refresh token présenté et en émet un nouveau. Si un jeton déjà consommé est présenté une deuxième fois, il a forcément été copié. Le service révoque alors *tous* les jetons de l'utilisateur, ce qui coupe aussi la session de l'attaquant.

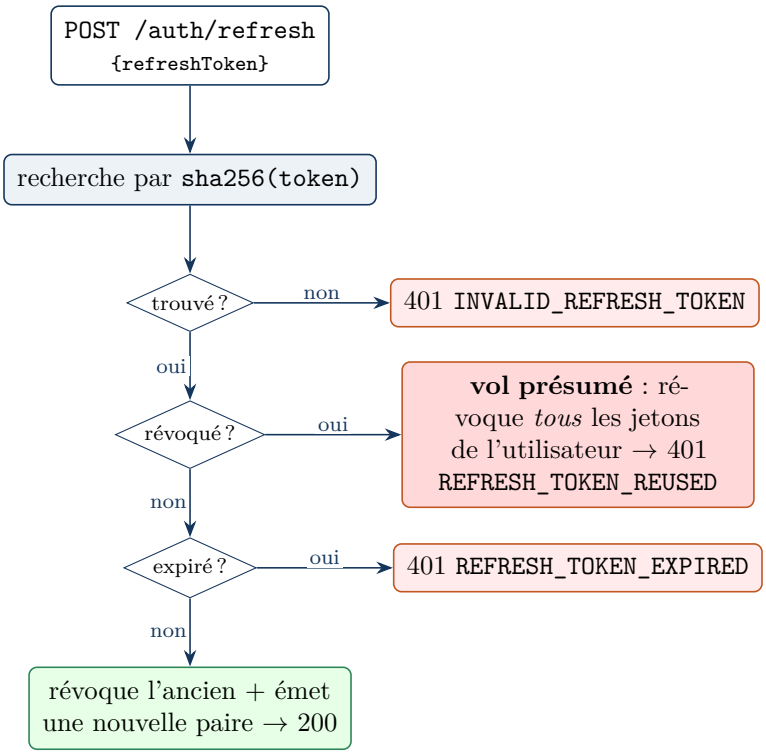


Figure 28 – Algorithme de `refresh()` dans `auth.service.ts`

Listing 6 – `auth.service.ts` : inscription et connexion

```
export async function register(input: RegisterInput) {
  const exists = await prisma.user.findUnique({ where: { email: input.email } })
  if (exists) throw new HttpError(409, 'Un compte existe deja avec cet e-mail', 'EMAIL_TAKEN')

  const user = await prisma.user.create({
    data: {
      firstName: input.firstName, lastName: input.lastName, email: input.email,
      passwordHash: await bcrypt.hash(input.password, config.bcryptRounds),   // coût 12
      role: 'STUDENT',
      student: { create: { filiere: input.filiere, enrolledYear: input.enrolledYear,
                           cpgeCenter: input.cpgeCenter, accessAllYears: true } },
    },
    include: withProfile,
  })
  return { user: toPublicUser(user), ...(await issueTokens(user.id, user.role)) }
}

export async function login(input: LoginInput) {
  const user = await prisma.user.findUnique({ where: { email: input.email }, include:
    withProfile })
  // compare() est appelé même si l'utilisateur n'existe pas (temps de réponse constant)
  const valid = await bcrypt.compare(input.password, user?.passwordHash ?? DUMMY_HASH)
  if (!user || !valid) throw new HttpError(401, 'E-mail ou mot de passe incorrect', '
    INVALID_CREDENTIALS')
  return { user: toPublicUser(user), ...(await issueTokens(user.id, user.role)) }
}
```

5.4.6 Mesures de sécurité

Menace	Parade mise en place
Vol de la base de données	bcrypt (coût 12) pour les mots de passe, SHA-256 pour les refresh tokens
Brute force sur la connexion	20 tentatives par tranche de 15 min et par IP sur login et register
Énumération des comptes	message d'erreur identique, `bcrypt.compare` exécuté même si l'e-mail n'existe pas
Usurpation d'identité	en-têtes `x-user-*` venant du client supprimés à la gateway
Appel direct d'un service	secret interne obligatoire, sinon 403
Vol d'un refresh token	rotation à chaque usage et révocation globale en cas de réutilisation
Fuite de données	`toPublicUser()` n'expose jamais `passwordHash` ; en-tête `x-powered-by` désactivé
Élévation de privilèges	le rôle vient du JWT signé ; `requireUser('ADMIN')` sur les routes d'administration
Entrées malformées	validation Zod de tous les corps de requête (e-mail normalisé, énumérations strictes)

Table 9 – Sécurité implémentée au Sprint 1

5.5 Les cinq autres microservices (squelettes)

Chaque squelette contient trois éléments :

1. un **schéma Prisma complet**, déjà poussé en base, qui traduit le diagramme de classes du CDC ;
2. un fichier `index.ts` qui **déclare toutes les routes prévues**. Elles répondent `501 NOT_IMPLEMENTED` avec le numéro du sprint concerné ;
3. le contrôle d'accès déjà en place sur les routes d'administration (`requireUser('ADMIN')`).

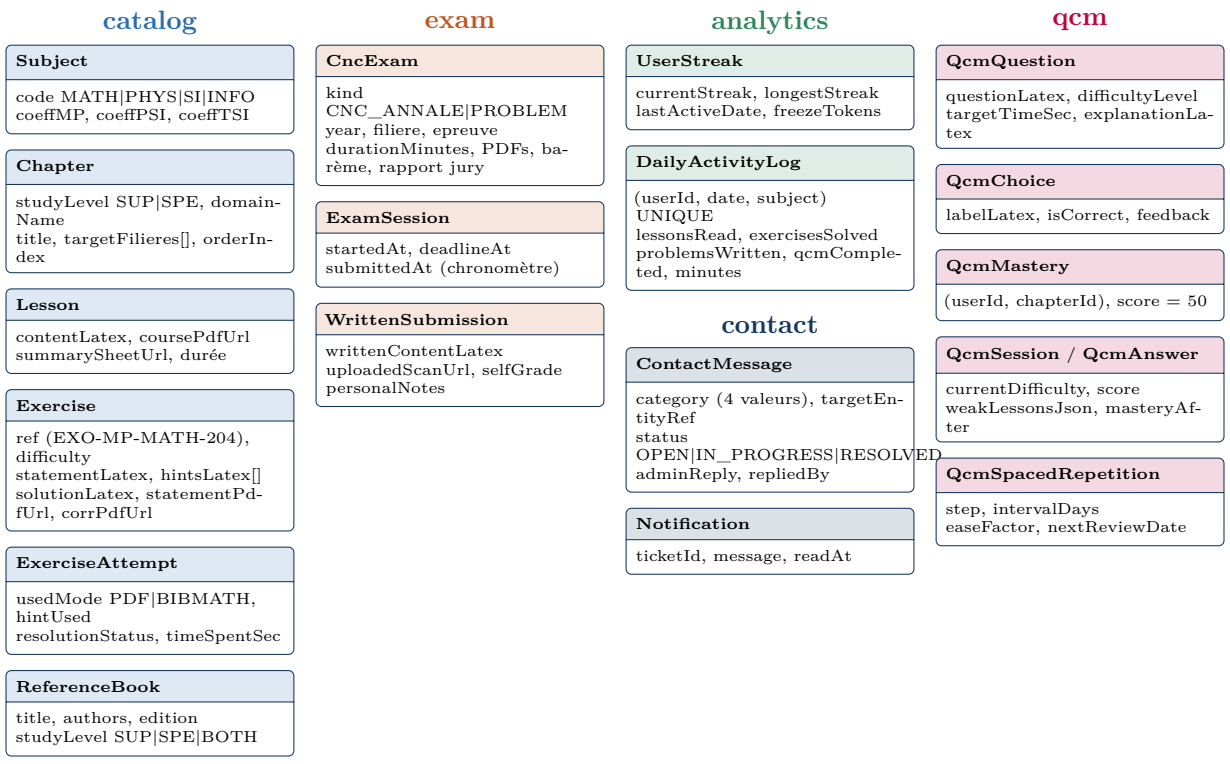


Figure 29 – Modèles Prisma des services squelettes (déjà créés en base)

Service	Routes déclarées (réponse 501 en attendant)
catalog (S2)	`GET /catalog/subjects`, `/chapters?subject&level&filiere`, `/chapters/:id`, `/lessons/:id`, `/lessons/:id/exercises`, `/exercises/:id`, `POST /exercises/:id/attempts`, `GET /books`, `/books/:id` ; admin : `POST|PATCH|DELETE /catalog/:entity`
exam (S3)	`GET /exams?kind&year&filiere&epreuve`, `/exams/:id`, `POST /exams/:id/sessions`, `POST /exams/sessions/:id/submit`, `PATCH /exams/submissions/:id`, `GET /exams/submissions/mine` ; admin : `POST|PATCH /exams`
analytics (S4)	`POST /analytics/activities`, `GET /analytics/me/streak`, `/me/heatmap?year`, `/me/subjects`, `/me/summary`
contact (S4)	`POST /contact/tickets`, `GET /tickets/mine`, `/tickets/:id`, `/notifications`, `PATCH /notifications/:id/read` ; admin : `GET /contact/admin/tickets`, `PATCH /admin/tickets/:id`, `POST /admin/tickets/:id/reply`
qcm (S5)	`POST /qcm/sessions`, `POST /sessions/:id/answers`, `POST /sessions/:id/finish`, `GET /qcm/flash`, `GET /qcm/mastery` ; admin : `POST|PATCH /qcm/questions`

Table 10 – Contrat d'API déjà fixé pour les sprints 2 à 5

5.6 Logique métier déjà codée : Streak et QCM adaptatif

Les deux algorithmes du CDC sont écrits comme des **fonctions pures**, sans base de données. Ils sont donc testables en isolation ; les sprints 4 et 5 n'auront qu'à les appeler depuis les routes.

5.6.1 Moteur de Streak (`analytics-streak-service/src/streak.ts`)

Le calcul suit le diagramme d'activité 4.3.1 du CDC. On note Δ le nombre de jours calendaires entre la dernière activité et aujourd'hui :

$$\text{currentStreak}' = \begin{cases} \text{currentStreak} & \text{si } \Delta = 0 \quad \text{(déjà actif aujourd'hui)} \\ \text{currentStreak} + 1 & \text{si } \Delta = 1 \\ 1 & \text{si } \Delta > 1 \end{cases}$$

$$\text{longestStreak}' = \max(\text{longestStreak},\ \text{currentStreak}')$$

Listing 7 – Fonction `updateStreak`

```
export function updateStreak(state: StreakState, today: string): StreakState {
  if (state.lastActiveDate === null) {
    return { currentStreak: 1, longestStreak: Math.max(1, state.longestStreak), lastActiveDate:
    today }
  }
  const delta = daysBetween(state.lastActiveDate, today)
  if (delta < 0) throw new Error('Activite anterieure a la derniere activite enregistree')
  if (delta === 0) return state
  const currentStreak = delta === 1 ? state.currentStreak + 1 : 1
  return { currentStreak, longestStreak: Math.max(state.longestStreak, currentStreak),
    lastActiveDate: today }
}
```

Le piège du fuseau horaire

Un « jour » doit être le jour *de l'étudiant* (Africa/Casablanca, UTC+1), pas celui du serveur. `localDate()` convertit un instant en date locale : une activité à 23h30 UTC le 4 novembre compte pour le **5** novembre à Casablanca. Un test unitaire couvre ce cas.

5.6.2 Moteur QCM adaptatif (`adaptive-qcm-service/src/adaptive-engine.ts`)

Formule (1) du CDC (section 6.2) :

$$M_{k+1}(u,c) = \text{clamp}_{[0,100]}\Big(M_k(u,c) + \alpha \cdot S_k \cdot \omega(D_k) \cdot \varphi(t_k)\Big)$$

$$S_k = \begin{cases} +1 & \text{réponse exacte} \\ -1{,}2 & \text{réponse fausse} \end{cases} \qquad \omega(D) = 1 + 0{,}25\,(D-1) \qquad \varphi(t) = \max\Big(0{,}5\,;\ \min\big(1{,}3\,;\ \tfrac{T_{\text{cible}}}{t}\big)\Big)$$

Le CDC ne fixe pas la valeur de α. Elle est réglée à $\alpha = 5$ (constante `ALPHA`) : une bonne réponse rapide en difficulté 4 rapporte au plus $5 \times 1{,}75 \times 1{,}3 \approx 11{,}4$ points.

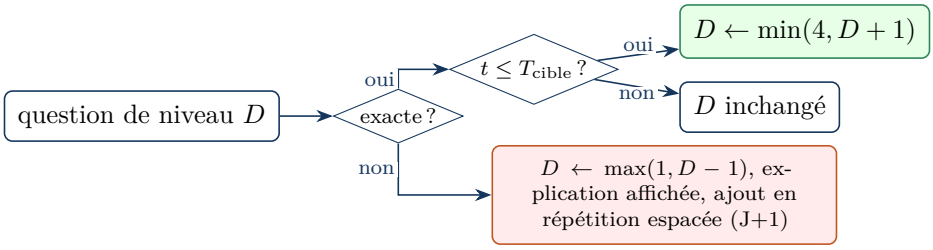


Figure 30 – Sélection adaptative de la difficulté (`nextDifficulty`)

Répétition espacée. Une question ratée entre dans la file avec une révision à J+1. Chaque bonne réponse lors d'une révision la fait avancer d'un cran ; une erreur la ramène à J+1.



5.7 Vérifications réalisées

Compilation et tests unitaires

- `npm run typecheck` : **0 erreur** sur les 8 workspaces (TypeScript `strict`).
- `npm test` : **15 tests réussis sur 15**. Ils couvrent le Streak (6 tests : $\Delta = 0$, $\Delta = 1$, $\Delta > 1$, changement de mois ou d'année, année bissextile, fuseau horaire), le moteur QCM (6 tests : bornes de M, de φ et de D, séquence J+1/J+3/J+7/J+21) et la validation de l'inscription (3 tests).
- Frontend : `next build` réussi après le déplacement (11 pages générées).

Test de bout en bout. PostgreSQL a été lancé dans Docker, les 6 schémas créés, puis les 7 processus démarrés. Le scénario ci-dessous a été joué contre la gateway (`http://localhost:4000`) :

Code	Requête	Comportement vérifié
201	`POST /api/auth/register`	compte créé, rôle STUDENT, filière PSI, `passwordHash` absent de la réponse
409	même e-mail une seconde fois	`EMAIL_TAKEN`
400	e-mail invalide, filière « BCPST »	`VALIDATION_ERROR` avec 7 champs détaillés
401	connexion avec un mauvais mot de passe	`INVALID_CREDENTIALS`

Code	Requête	Comportement vérifié
200	connexion correcte	paire de jetons
401	`GET /api/users/me` sans jeton	bloqué par la gateway
401	`GET /api/users/me` avec un jeton forgé	`INVALID_TOKEN`
200	`GET /api/users/me` avec un jeton valide	profil renvoyé
200	`PATCH /api/users/me {enrolledYear: SPE}`	profil mis à jour
403	`GET /api/users` en tant qu'étudiant	réservé à ADMIN
403	même requête + en-tête `x-user-role: ADMIN`	**usurpation bloquée**
200	`GET /api/users` en tant qu'admin	liste paginée (3 comptes)
200	`POST /api/auth/refresh`	rotation : nouvelle paire
401	ré-utilisation de l'ancien refresh token	`REFRESH_TOKEN_REUSED`
401	refresh avec le jeton issu de la rotation	révoqué aussi (révocation globale)
204	`POST /api/auth/logout`	jeton révoqué
501	`GET /api/catalog/subjects`	« sera implémenté au Sprint 2 »
403	appel direct `http://localhost:4001/users/me`	refusé sans secret interne
200	`GET /api/health`	gateway + 6 services « up »

Table 11 – Scénario de bout en bout : 19 cas, tous conformes

6 Ce qu'il reste à faire

6.1 Backend : sprints restants

Le contrat d'API et les schémas existent déjà. Pour chaque sprint, le travail consiste à remplacer `notImplemented(...)` par de vrais handlers, en suivant le modèle de `auth-user-service` : `schemas.ts` (Zod), `*.service.ts` (règles métier), `routes.ts`.

Sprint	Service	Travail restant
S2 sem. 3–5	catalog-course	Lecture des matières, chapitres (filtres niveau et filière, *sans* restriction d'accès), leçons, exercices double mode ; enregistrement d'une `ExerciseAttempt` qui publie un événement d'activité ; CRUD admin ; section Ouvrages (filtres matière, filière, niveau) ; script de *seed* qui importe les 26 chapitres de `COURSES_DATABASE` du frontend ; envoi des PDF de cours vers MinIO (URL présignées).
S3 sem. 6–8	exam-submission	Import des 437 épreuves depuis les manifestes JSON du frontend ; filtres année, filière, épreuve ; `ExamSession` dont la fin (`deadlineAt`) est calculée côté serveur (anti-triche) ; soumission LaTeX ou scan (MinIO) ; corrigé et barème renvoyés *uniquement* après soumission ; auto-évaluation ; portefeuille « Mes Problèmes Rédigés » (UC07).

Sprint	Service	Travail restant
S4 sem. 9–10	analytics-streak	`POST /analytics/activities` : *upsert* de `DailyActivityLog` et appel à `updateStreak()` dans **une même transaction** ; heatmap annuelle, radar par matière (ventilé Sup/Spé), compteurs ; cache Redis du Dashboard (objectif : moins de 200 ms au 95e centile).
S4	admin-contact	Création de ticket avec `targetEntityRef` pré-rempli et l'une des 4 catégories ; cycle OPEN → IN_PROGRESS → RESOLVED ; réponse de l'administrateur qui crée une `Notification` et envoie un e-mail (SMTP, par exemple Nodemailer).
S5 sem. 11–12	adaptive-qcm	Session adaptative qui s'appuie sur `updateMastery()`, `nextDifficulty()` et `scheduleReview()` ; « QCM Flash Quotidien » (questions dont la révision est due) ; bilan `weakLessonsJson` ; événement Streak ; CRUD admin des questions.
Transverse	tous	Communication entre services par événements Redis (par exemple `activity.recorded`) plutôt que par appels HTTP directs ; migrations Prisma versionnées (`prisma migrate`) au lieu de `db push` ; journaux structurés ; tests d'intégration par service ; Dockerfile par service ; documentation OpenAPI.

Table 12 – Travail backend restant, par sprint (planning du CDC, section 7.2)

6.1.1 Flux d'événements prévu pour le Streak

Le CDC demande que *toute* activité pédagogique valide le Streak (UC09). Plusieurs services produisent ces activités, mais seul `analytics-streak-service` doit tenir le compteur :

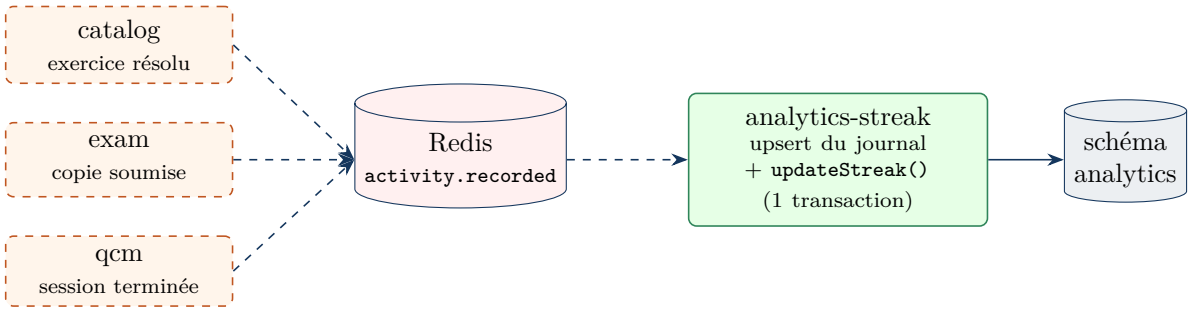


Figure 31 – Publication et consommation des événements d'activité (à implémenter)

6.2 Frontend : branchement sur l'API

6.2.1 Étape 1 : un client HTTP unique

Il faut créer `frontend/lib/api.ts`. Ce module ajoute l'en-tête `Authorization`, appelle `/api/auth/refresh` une seule fois quand il reçoit un 401, puis rejoue la requête. Il remplacera les fonctions de `lib/auth.ts`. Le refresh token devrait idéalement être stocké dans un cookie `httpOnly` plutôt que dans `localStorage`.

Listing 8 – Esquisse de `frontend/lib/api.ts` (à écrire)

```
1  const API = process.env.NEXT_PUBLIC_API_URL ?? 'http://localhost:4000/api'
2
3  export async function api<T>(path: string, init: RequestInit = {}, retry = true): Promise<T> {
```

```
  const res = await fetch(API + path, {
    ...init,
    headers: { 'Content-Type': 'application/json',
               Authorization: `Bearer ${getAccessToken()}`, ...init.headers },
  })
  if (res.status === 401 && retry && (await refreshTokens())) return api<T>(path, init, false)
  if (!res.ok) throw await res.json()          // { error, message, details? }
  return res.status === 204 ? (undefined as T) : res.json()
}
```

6.2.2 Étape 2 : remplacer les données en dur

Emplacement dans le frontend	Donnée simulée	Endpoint cible
`lib/auth.ts`, `LoginPage.tsx`, `SignUpPage.tsx`	`DEMO_STUDENT`, faux jetons, mot de passe ignoré	`/api/auth/*` — **disponible**
`app/parametres/page.tsx`	profil dans `localStorage`, faux changement de mot de passe	`PATCH /api/users/me` — **disponible**
`app/admin/page.tsx`	aucune vérification du rôle	`GET /api/users/me` (rôle) — **disponible**
`app/page.tsx` (dashboard), `ActivityBar`, `StreakBadge`, `AchievementBadgesCard`	72 %, 12 jours, 254 h, heatmap pseudo-aléatoire	`/api/analytics/me/*`
`CourseExerciseQcmDoubleMode.tsx`, `COURSES_DATABASE`, `SUBJECTS`	26 chapitres, 28 exercices	`/api/catalog/*`
même fichier : `exerciseEvaluations`, « Signaler »	auto-évaluation et erratum non conservés	`POST /api/catalog/exercises/:id/attempts`, `POST /api/contact/tickets`
même fichier : QCM flash et banque QCM	maîtrise +6/−4, chronomètre figé	`/api/qcm/*`
`lib/concours-annales-database.ts` et manifestes JSON	137 épreuves, URL statiques	`/api/exams`
`ConcoursPdfExamViewer.tsx`	remise de copie et infractions en mémoire, candidat en dur	`/api/exams/:id/sessions`, `/submit`
`ExercisePdfTemplate.tsx`, `lib/classiques-problems.ts`	10 problèmes, notes locales	`/api/exams?kind=PROBLEM`
`ContactAdminTemplate.tsx`	tickets en dur, réponse automatique au bout de 1,4 s	`/api/contact/*`
`AdminBackOffice.tsx`	catalogue, annales, QCM et conversations en état local	routes admin des 4 services
`PochettesWorkspace.tsx`	dossiers et pages personnels	*à arbitrer* (aucun service prévu par le CDC)

Table 13 – Données simulées à remplacer par des appels à l'API

6.2.3 Étape 3 : harmoniser les énumérations

Champ	Frontend actuel	Backend (CDC)
Filière	`'MP*' | 'MP' | 'PSI*' | 'PSI' | 'TSI'`	`MP | PSI | TSI`
Année	`'1re année Sup' | '2e année Spé'`	`SUP | SPE`
Niveau d'un chapitre	`'sup' | 'spe'`	`SUP | SPE`
Matière	`maths, physique, si (ou sii), info`	`MATH | PHYS | SI | INFO`
Auto-évaluation	`mastered | with_help | review`	`MASTERED | WITH_HINT | TO_REVIEW`
Statut de ticket	`pending | in_progress | resolved`	`OPEN | IN_PROGRESS | RESOLVED`

Table 14 – Correspondances à mettre en place (par exemple dans `frontend/lib/mappers.ts`)

Pour la filière, il faut décider si l'« étoile » (MP*) doit être conservée. Si oui, il faudrait ajouter un champ booléen `starClass` au modèle `Student`.

6.3 Écarts restants avec le CDC

Exigence du CDC	Frontend	Backend
Inscription et connexion JWT + bcrypt (M1)	**Partiel** (simulé)	**Fait**
Basculeur « Tous les niveaux / Sup / Spé »	**Partiel** (pas de « Tous »)	**Fait** (`accessAllYears`)
Organisation domaine → chapitre → leçon (M2)	**Partiel**	**Fait** (schéma)
Exercices en mode PDF avec énoncé et corrigé séparés	**À faire** (planche HTML)	**Fait** (schéma)
Annales 2010–2026, filtre par filière (M3)	**Partiel** (MP seulement)	**À faire**
« Mes Problèmes Rédigés » : soumission, archive, barème	**À faire**	**À faire**
Bibliothèque d'ouvrages (M4)	**À faire** (absente)	**À faire** (schéma prêt)
Record de Streak, ventilation Sup/Spé (M5)	**À faire**	**À faire** (algorithme prêt)
Signalement contextualisé et 4 catégories (M6)	**Partiel**	**À faire** (schéma prêt)
QCM adaptatif et répétition espacée (P15)	**Partiel**	**À faire** (algorithmes prêts)
Modules optionnels M7 (TIPE, simulateur de rang, oraux…)	**À faire**	**À faire**

Table 15 – Couverture du CDC par couche

6.4 Dette technique du frontend à traiter

1. **Découper les gros composants** : extraire les données de `CourseExerciseQcmDoubleMode.tsx` (5 276 lignes) dans `lib/`, puis séparer les sous-vues (`CoursView`, `TdView`, `QcmBank`, `AnnalesPortal`). Faire de même pour `AdminBackOffice.tsx`.
2. **Protéger les routes** : rediriger vers `/login` sans session ; contrôler le rôle ADMIN sur `/admin` (le backend refuse déjà, mais l'interface s'affiche quand même).
3. **Déconnexion réelle** : appeler `POST /api/auth/logout`, puis `clearUserSession()`.
4. **Supprimer le code mort** : `StudentDrawer`, `PersonalWorkspaceModal`, `glass-folder-cards`, `ui/button`, `@base-ui/react`.
5. **Réactiver la vérification de types** : retirer `ignoreBuildErrors` de `next.config.mjs` et corriger les erreurs.
6. **Corriger l'échappement LaTeX** : certaines formules contiennent `'\forall'` au lieu de `'\\forall'` (en JavaScript, `\f` devient un caractère de contrôle).

7. **Déplacer les 570 Mo de PDF** de `public/` vers MinIO, servis par `exam-submission-service` avec des URL signées.

6.5 Planning proposé

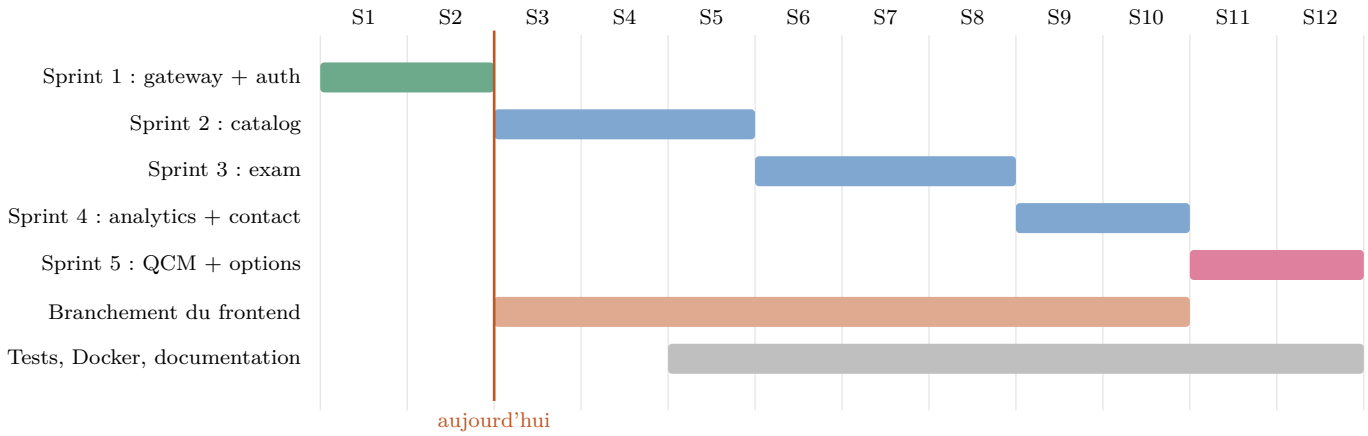


Figure 32 – Avancement par rapport au planning du CDC (le Sprint 1 est terminé)

7 Guide de démarrage

7.1 Prérequis

Node.js 24 ou plus récent, npm 11, et Docker Desktop (pour PostgreSQL, Redis et MinIO).

7.2 Backend

```
cd backend
copy .env.example .env      # puis remplir JWT_ACCESS_SECRET et INTERNAL_SECRET
                            # node -e "console.log(require('crypto').randomBytes(48).toString('
    hex'))"
npm install
npm run infra:up            # PostgreSQL :5434, Redis :6379, MinIO :9000 (console :9001)
npm run db:generate
npm run db:push
npm run db:seed             # admin@learning-cnc.ma / AdminPassword2026!
                            # achraf.lemrani@cpge-cnc.ma / StudentPassword2026!
npm run dev                 # gateway :4000 + 6 services :4001..4006
```

Vérification rapide :

```
curl http://localhost:4000/api/health
curl -X POST http://localhost:4000/api/auth/login -H "Content-Type: application/json" \
     -d "{\"email\":\"admin@learning-cnc.ma\",\"password\":\"AdminPassword2026!\"}"
```

7.3 Frontend

```
cd frontend
npm install                 # postinstall : copie les fichiers de pdf.js dans public/pdfjs
npm run dev                 # http://localhost:3000 (ou 3001 si le port 3000 est pris)
```

7.4 Ajouter une route dans un service (méthode)

1. Écrire le schéma Zod de l'entrée dans `src/schemas.ts`.
2. Écrire la fonction métier dans `src/<domaine>.service.ts`. Elle utilise `prisma` et lève une `HttpError` en cas de règle violée.
3. Dans `src/routes.ts` (ou `index.ts`), remplacer `todo` par :

```
router.get('/catalog/chapters', async (req, res) => {
  res.json(await catalog.listChapters(chapterQuerySchema.parse(req.query)))
})
```

4. Si le schéma Prisma a changé : `npm run db:generate` puis `npm run db:push`.
5. Ajouter un test (`*.test.ts`), puis lancer `npm run typecheck` et `npm test`.

Pour relire le code dans le bon ordre

1. `backend/gateway/src/index.ts`, puis `authenticate.ts` : comment une requête entre.
2. `backend/packages/common/src/` : le socle de tous les services.
3. `backend/services/auth-user-service/src/routes.ts`, puis `auth.service.ts` et `tokens.ts` : un service complet.
4. `prisma/schema.prisma` de chaque service : le modèle de données du CDC.
5. `streak.ts` et `adaptive-engine.ts`, avec leurs tests : les algorithmes.
6. `frontend/app/page.tsx`, puis `components/CourseExerciseQcmDoubleMode.tsx` : l'interface.